Spring is a dependency injection framework!

https://github.com/in28minutes/spring-master-class

What is dependency injection?
What is dependency ?

Structure in which we develop our web application:

Web layer(concered with UI of web applic) depend on business logic and business layer depend on some class present inside data .

Eg→ //here we are directly instantiating the sortAlgo ,If I have want to use the other sorting algo then what?? I have to modify the code .This code is called Tight Coupling!This is not good code.

```
Public class ComplexBusinessServ{

    SortAlgorithm sortAlgorithm=new BubbleSortAlgorithm();//direct instan…

    Public class BubbleSortAlgorith implements SortAlgorithm {
```

//Loose coupling is considered to be good.Here we remove the instantiation of Dependencies.

We provide a constructor or setter with Dependency as argument—>//loose coupling

```
        SortAlgorithm sortAlgorithm=new BubbleSortAlgorithm();
ComplexBusinessServ businessService=
                        new ComplexBusinessService(sortAlgorithm);
```

Here we creating new instance of sortALgo and a instance of complexBusinessServ.

This code does->Creating object (bubb..) and populating dependencies(sortA..)

Spring Framework does exactly the job you're seeing in the code.Here it instantiate objects and populate the dependencies.It is your job as a programmer to tell the Spring framework -
What are the objects it would need to manage and what are the dependencies of each class?

@Component
public class ComplexBusinessService {

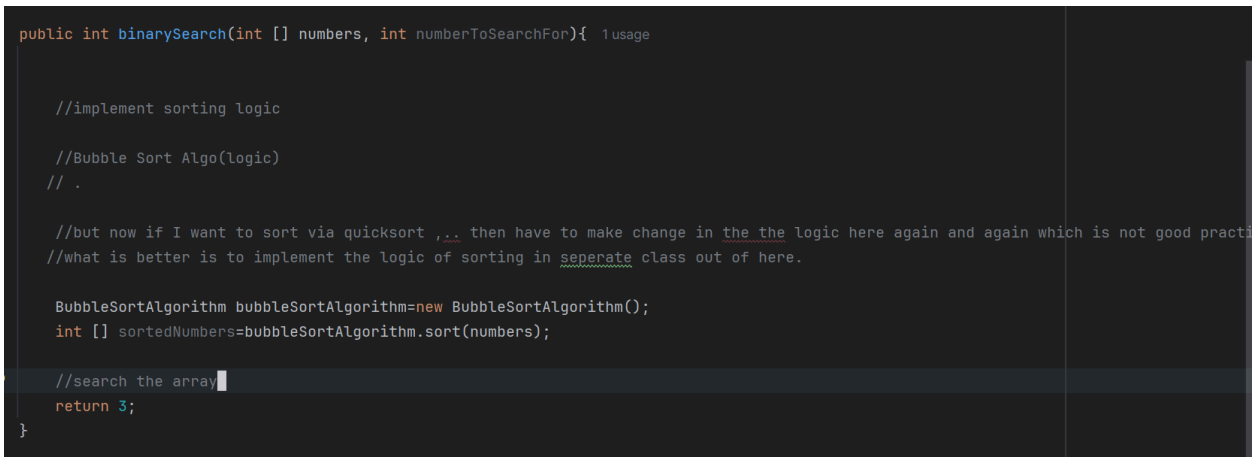
```
    @Autowired
    SortAlgorithm sortAlgorithm;
}

@Component
public class BubbleSortAlgorithm implements SortAlgorithm {
    // Bubble sort implementation here
}
```

Spring Boot will create instances of `ComplexBusinessService` and `BubbleSortAlgorithm` for you.

When `ComplexBusinessService` is created, Spring will automatically inject the `BubbleSortAlgorithm` instance into the `sortAlgorithm` field because it's marked with `@Autowire`

Binary Search Eg–>



```
public int binarySearch(int [] numbers, int numberToSearchFor){  1 usage


    //implement sorting logic

    //Bubble Sort Algo(logic)
   // .

    //but now if I want to sort via quicksort ,.. then have to make change in the the logic here again and again which is not good practi
   //what is better is to implement the logic of sorting in seperate class out of here.

    BubbleSortAlgorithm bubbleSortAlgorithm=new BubbleSortAlgorithm();
    int [] sortedNumbers=bubbleSortAlgorithm.sort(numbers);

    //search the array
    return 3;
}
```

Better to move the logic of sorting algo . out of here ,make a different class.

```java
package com.example.FirstSpringproject;

public class BubbleSortAlgorithm {  2 usages

    public int [] sort(int [] numbers){  1 usage
        //Logic for Bubble Sort

        //so if any time if the logic of this sorting algo change make updation here only.

        return numbers;
    }
}
```

FirstSpringprojectApplication.java | BinarySearchImpl.java | QuickSortAlgorithm.java

```java
package com.example.FirstSpringproject;

public class QuickSortAlgorithm {  no usages

    public int[] sort(int [] numbers){  no usages

        //logic of quick sort
        return numbers;
    }
}
```

But how to move dynamically between these algo. Of sorting??

##Now quickSort and Bubble sort both are implementing the interface SortAlgorithm

```java
package com.example.FirstSpringproject;

public class QuickSortAlgorithm implements SortAlgorithm{  no usages

    public int[] sort(int [] numbers){  1 usage

        //logic of quick sort
        System.out.println("QuickSort");    return numbers;
    }
}
```

FirstSpringprojectApplication.java | BinarySearchImpl.java | QuickSortAlgorithm.java | BubbleSortAlgorithm.java

```java
package com.example.FirstSpringproject;

public class BubbleSortAlgorithm implements SortAlgorithm {  no usages

    public int [] sort(int [] numbers){  1 usage
        //Logic for Bubble Sort

        //so if any time if the logic of this sorting algo change make updation here only.
        System.out.println("BubbleSort");
        return numbers;

    }
}
```

FirstSpringprojectApplication.java | SortAlgorithm.java × | BinarySearchImp

```java
package com.example.FirstSpringproject;

public interface SortAlgorithm {  4 usages  2 implementations
    public int[] sort(int [] numbers);  1 usage  2 implementations
}
```

FirstSpringprojectApplication.java | BinarySearchImpl.java | QuickSortAlgorithm.java | Bub

```
5
6   public class BinarySearchImpl {  2 usages
7   private SortAlgorithm sortAlgorithm;  3 usages
8
9       public BinarySearchImpl(SortAlgorithm sortAlgorithm) {  1 usage
10          this.sortAlgorithm = sortAlgorithm;
11      }
12
13
14      public int binarySearch(int [] numbers,int numberToSearchFor){  1 usage
15
16        //BubbleSortAlgorithm bubbleSortAlgorithm=new BubbleSortAlgorithm();
17          int [] sortedNumbers=sortAlgorithm.sort(numbers);
18         System.out.println(sortAlgorithm);
19          //search the array
20          return 3;
21
22  }
23  }
```

FirstSpringprojectApplication.java | BinarySearchImpl.java | QuickSortAlgorithm.java | BubbleSortAlgorithm.java

```
package com.example.FirstSpringproject;

import ...

@SpringBootApplication
public class FirstSpringprojectApplication {

    public static void main(String[] args) {


        BinarySearchImpl binarySearch=new BinarySearchImpl( new QuickSortAlgorithm());

        int result=binarySearch.binarySearch(new int[] {12,4,6}, numberToSearchFor: 3);
        System.out.println(result);


        SpringApplication.run(FirstSpringprojectApplication.class, args);
    }

}
```

FirstSpringprojectApplication.java | BinarySearchImpl.java | QuickSortAlgorithm.java | BubbleSortAlgorithm.java

```java
package com.example.FirstSpringproject;

import ...

@SpringBootApplication
public class FirstSpringprojectApplication {

    public static void main(String[] args) {

        //we are creating the object of algo and binary search  here and we are wiring them together
        BinarySearchImpl binarySearch=new BinarySearchImpl( new QuickSortAlgorithm());

        // we are invoking the binary search
        int result=binarySearch.binarySearch(new int[] {12,4,6}, numberToSearchFor: 3);
        System.out.println(result);


        SpringApplication.run(FirstSpringprojectApplication.class, args);
    }

}
```

Here in eg now we do it using the Spring FrameWork.:

FirstSpringprojectApplication.java | application.properties | BinarySearchImpl.java | QuickSortAlgorithm.java | BubbleSortA

```java
package com.example.FirstSpringproject;

import ...

@SpringBootApplication
public class FirstSpringprojectApplication {

    //using Spring->

    //What are the beans? BinarySeIml,BubbleSort
    //What are the dependencies of a bean? //SortAlgo
    //Where to search for beans? No need because we are in same package
    public static void main(String[] args) {

        //we are creating the object of algo and binary search  here and we are wiring them together
        //BinarySearchImpl binarySearch=new BinarySearchImpl( new QuickSortAlgorithm());
          //Application context will maintain all beans
        // we are invoking the binary search

        ApplicationContext applicationContext = SpringApplication.run(FirstSpringprojectApplication.class, args);
        BinarySearchImpl binarySearch=applicationContext.getBean(BinarySearchImpl.class);
        int result=binarySearch.binarySearch(new int[] {12,4,6}, numberToSearchFor: 3);
        System.out.println(result);
    }
//output->BubbleSort
//com.example.FirstSpringproject.BubbleSortAlgorithm@aa5455e
//3
}
```

FirstSpringprojectApplication.java | application.properties | BinarySearchImpl.java

```java
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

@Component  2 usages
public class BinarySearchImpl {
@Autowired  3 usages
private SortAlgorithm sortAlgorithm;
    public BinarySearchImpl(SortAlgorithm sortAlgorithm) {  1 usage
        this.sortAlgorithm = sortAlgorithm;
    }


   public int binarySearch(int [] numbers,int numberToSearchFor){  1 usage

     //BubbleSortAlgorithm bubbleSortAlgorithm=new BubbleSortAlgorithm();
      int [] sortedNumbers=sortAlgorithm.sort(numbers);
     System.out.println(sortAlgorithm);
      //search the array
      return 3;

}
}
```

```java
package com.example.FirstSpringproject;

import org.springframework.stereotype.Component;

@Component  no usages
public class BubbleSortAlgorithm implements SortAlgorithm {

    public int [] sort(int [] numbers){  1 usage
        //Logic for Bubble Sort

        //so if any time if the logic of this sorting algo change make updation here only.
        System.out.println("BubbleSort");
        return numbers;

    }
}
```

```java
package com.example.FirstSpringproject;

public class QuickSortAlgorithm implements SortAlgorithm{  no usages

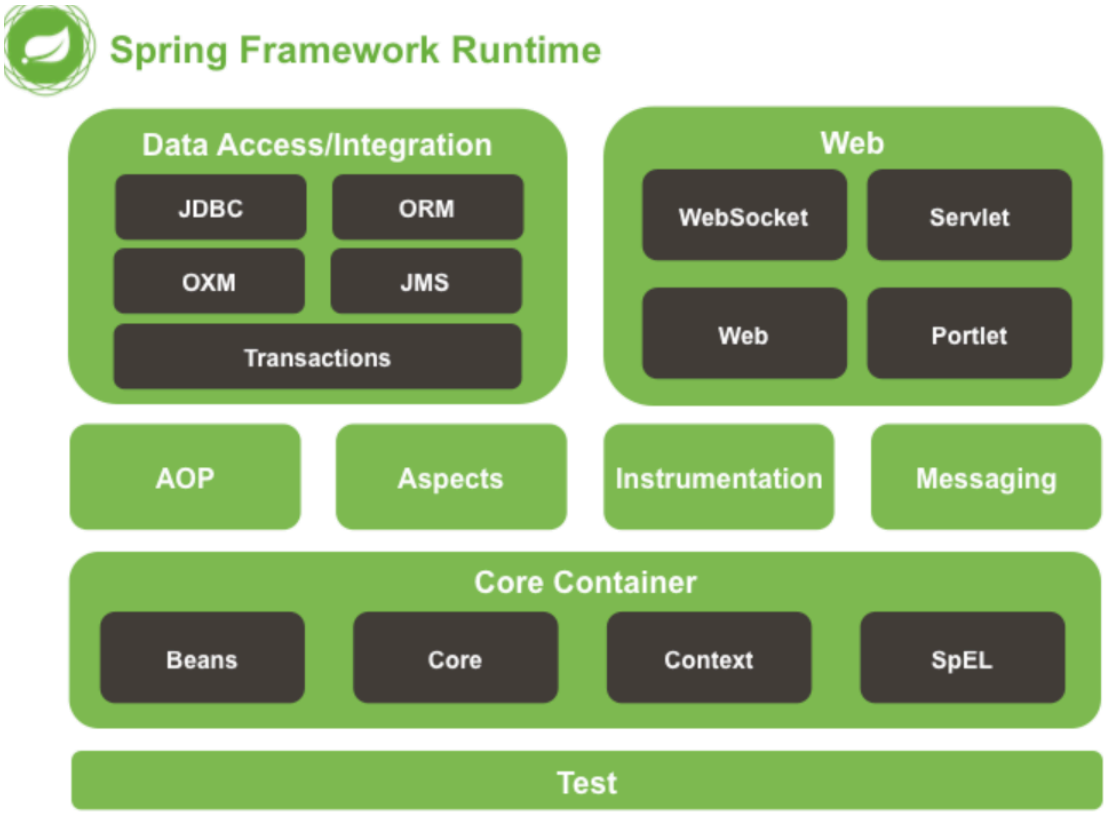
    public int[] sort(int [] numbers){  1 usage

        //logic of quick sort
        System.out.println("QuickSort");  return numbers;
    }
}
```

Constructor using->

```java
public BinarySearchImpl(SortAlgorithm sortAlgorithm) {
    this.sortAlgorithm = sortAlgorithm;
}
```

Different modules offered by Spring FrameWork->
It has solution for Data Layer,Web layer,Cross cutting concerns..



Cross cutting concerns-> unit testing

Autowiring:

If there are more than one Componets,then how to specify which to use(Autowire)??

->1) Using @Primary anotation

2)Specifying the name of algo like this->

```
@Component
public class BinarySearchImpl {

    @Autowired
    private SortAlgorithm bubbleSortAlgorithm;

    public int binarySearch(int[] numbers, int numberToSearchFor) {

        int[] sortedNumbers = bubbleSortAlgorithm.sort(numbers);
        System.out.println(bubbleSortAlgorithm);
        // Search the array
        return 3;
    }
```

Among both @Primary has higher priority then methodname. 1st method >>> 2nd method

3)Using

@Qualifier(“quick”)

```
@Component  no usages
@Qualifier("quick")
public class QuickSortAlgorithm implements SortAlgorithm{

    public int[] sort(int [] numbers){  1 usage

        //logic of quick sort
        System.out.println("QuickSort");   return numbers;
    }
}
```

```java
import org.springframework.stereotype.Component;

@Component  2 usages
public class BinarySearchImpl {

    @Autowired  3 usages
    @Qualifier("quick")
    private SortAlgorithm sortAlgorithm;
//    public BinarySearchImpl(SortAlgorithm sortAlgorithm) {
//        this.sortAlgorithm = sortAlgorithm;
//    }
```

Bean Scope:
1)SingleTon—>

```java
        var binarySearch = applicationContext.getBean(BinarySearchImpl.class);

        var binarySearch1 = applicationContext.getBean(BinarySearchImpl.class);

        System.out.println(binarySearch);
        System.out.println(binarySearch1);
```

Get same bean->the hashcodes are same

@1e81f160
@1e81f160

2)Prototype->

```java
@Component
@Scope("prototype")
public class BinarySearchImpl {

    @Autowired
    @Qualifier("bubble")
    private SortAlgorithm sortAlgorithm;

    public int binarySearch(int[] numbers, int numberToSearchFor) {

        int[] sortedNumbers = sortAlgorithm.sort(numbers);
        System.out.println(sortAlgorithm);
        // Search the array
        return 3;
    }
```

Hardcoding prototype is not good practice.

```
 8
 9 @Component
10 @Scope(ConfigurableBeanFactory.SCOPE_PROTOTYPE)
11 public class BinarySearchImpl {
12
13     @Autowired
```

Getting two different instances->

```
com.in28minutes.spring.basics.springin5steps.BinarySearchImpl@421e361
com.in28minutes.spring.basics.springin5steps.BinarySearchImpl@59d4cd39
com.in28minutes.spring.basics.springin5steps.BubbleSortAlgorithm@389c4eb1
3
```

In personDAO eg->
output->2diff instances because we specified the scope prototype

```
ing beans for JMX exposure on startup
 SpringIn5StepsScopeApplication in 2.544 seconds (JVM running for 3.774)
minutes.spring.basics.springin5steps.scope.PersonDAO@55c53a33
minutes.spring.basics.springin5steps.scope.JdbcConnection@5a37d3ed
minutes.spring.basics.springin5steps.scope.PersonDAO@26df6e3a
minutes.spring.basics.springin5steps.scope.JdbcConnection@5a37d3ed
```

```
package com.in28minutes.spring.basics.springin5steps.scope;

import org.springframework.beans.factory.config.ConfigurableBeanFactory;
import org.springframework.context.annotation.Scope;
import org.springframework.context.annotation.ScopedProxyMode;
import org.springframework.stereotype.Component;

@Component
@Scope(value=ConfigurableBeanFactory.SCOPE_PROTOTYPE,
                proxyMode = ScopedProxyMode.TARGET_CLASS)
public class JdbcConnection {
        public JdbcConnection() {
                System.out.println("JDBC Connection");
        }
}
```

```
package com.in28minutes.spring.basics.springin5steps.scope;
```

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

@Component
public class PersonDAO {

	@Autowired
	JdbcConnection jdbcConnection;

	public JdbcConnection getJdbcConnection() {
		return jdbcConnection;
	}

	public void setJdbcConnection(JdbcConnection jdbcConnection) {
		this.jdbcConnection = jdbcConnection;
	}
}
```

/src/main/java/com/in28minutes/spring/basics/springin5steps/SpringIn5StepsScopeApplication.java

```
package com.in28minutes.spring.basics.springin5steps;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;

import com.in28minutes.spring.basics.springin5steps.scope.PersonDAO;

@SpringBootApplication
public class SpringIn5StepsScopeApplication {

	private static final Logger LOGGER = LoggerFactory.getLogger(SpringIn5StepsScopeApplication.class);

	public static void main(String[] args) {

		var applicationContext = SpringApplication.run(SpringIn5StepsScopeApplication.class, args);

		var personDao = applicationContext.getBean(PersonDAO.class);

		var personDao2 = applicationContext.getBean(PersonDAO.class);

		LOGGER.info("{}", personDao);
		LOGGER.info("{}", personDao.getJdbcConnection());

		LOGGER.info("{}", personDao2);
		LOGGER.info("{}", personDao.getJdbcConnection());

	}
}
```

```java
@PostConstruct
public void postConstruct() {
    logger.info("postConstruct");
}

@PreDestroy
public void preDestroy() {
    logger.info("preDestroy");
}
```

##

```xml
<dependency>
    <groupId>javax.inject</groupId>
    <artifactId>javax.inject</artifactId>
    <version>1</version>
</dependency>
```

##

```xml
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-core</artifactId>
</dependency>

<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
</dependency>
```

Shifting from SpringBoot to Spring core—->remoce spring starter dependency and add spring core and spring context in pom.xml

1) Remove @SpringBootApplication and write @Configuration->it define that this is spring configur.
2) Dont use SpringApplication.run instead create applic context as->

   new AnnotationConfigurationContext (_)
3) Add @ComponetScan →spring not do it itself, its feature of springboot
4) Add dependency->org.slf4j

```
<dependency>
    <groupId>org.slf4j</groupId>
    <artifactId>slf4j-api</artifactId>
</dependency>
```

5) Logger to work-> add dependency->

```
<dependency>
    <groupId>ch.qos.logback</groupId>
    <artifactId>logback-classic</artifactId>
</dependency>
```

6)AnnotationConfContext ko close krna ha using.close() or try catch

#Defining Spring Application context using xml

(Without using annotations)

We first have to define the beans->

XmlJdbcConnection.java | XmlPersonDAO.java | applicationContext.xml | SpringIn5StepsXMLContextApplication.java

```java
package com.in28minutes.spring.basics.springin5steps.xml;

public class XmlJdbcConnection {
    public XmlJdbcConnection() {
        System.out.println("JDBC Connection");
    }
}

```

XmlJdbcConnection.java | XmlPersonDAO.java | applicationContext.xml | SpringIn5StepsXMLContextApplication.java

```java
package com.in28minutes.spring.basics.springin5steps.xml;

public class XmlPersonDAO {

    XmlJdbcConnection xmlJdbcConnection;

    public XmlJdbcConnection getXmlJdbcConnection() {
        return xmlJdbcConnection;
    }

    public void setXmlJdbcConnection(XmlJdbcConnection jdbcConnection) {
        this.xmlJdbcConnection = jdbcConnection;
    }
}

```

Then define the dependencies in .xml
(become difficult to manage ,because each time have to add them manually in .xml)

XmlJdbcConnection.java | XmlPersonDAO.java | applicationContext.xml | SpringIn5StepsXMLContextApplication.java

```xml
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="xmlJdbcConnection"
    class="com.in28minutes.spring.basics.springin5steps.xml.XmlJdbcConnection">
    </bean>

    <bean id="xmlPersonDAO" class="com.in28minutes.spring.basics.springin5steps
        <property name="xmlJdbcConnection" ref="xmlJdbcConnection"/>
    </bean>

</beans>
```

/src/main/java/com/in28minutes/spring/basics/springin5steps/SpringIn5StepsXMLContextApplication.java

```java
package com.in28minutes.spring.basics.springin5steps;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.support.ClassPathXmlApplicationContext;

import com.in28minutes.spring.basics.springin5steps.xml.XmlPersonDAO;

@Configuration
@ComponentScan
public class SpringIn5StepsXMLContextApplication {

	public static void main(String[] args) {

		try (var applicationContext = new ClassPathXmlApplicationContext("applicationContext.xml")) {

			var personDao = applicationContext.getBean(XmlPersonDAO.class);
			System.out.println(personDao);
			System.out.println(personDao.getXmlJdbcConnection());
		}
	}
}
```

Using component scan ->

```
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:context="http://www.springframework.org/schema/context"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/context
        http://www.springframework.org/schema/context/spring-context.xsd">

    <context:component-scan base-package="com.in28minutes.spring.basics"/>

    <bean id="xmlJdbcConnection"
    class="com.in28minutes.spring.basics.springin5steps.xml.XmlJdbcConnection">
    </bean>

    <bean id="xmlPersonDAO" class="com.in28minutes.spring.basics.springin5steps
        <property name="xmlJdbcConnection" ref="xmlJdbcConnection"/>
    </bean>
```

IOC Controller->

```
@RestController
public class WelcomeController {

    private WelcomeService service = new WelcomeService();

  @RequestMapping("/welcome")
  public String welcome() {
    return service.retrieveWelcomeMessage();
  }
}
```

WITH SPRING

```
@Component
public class WelcomeService {
    //Bla Bla Bla
}

@RestController
public class WelcomeController {

    @Autowired
    private WelcomeService service;

  @RequestMapping("/welcome")
  public String welcome() {
    return service.retrieveWelcomeMessage();
  }
```

Spring Boot

Spring Boot is a framework for building applications in Java.It provide the tools ,with the help of which we can easily make Spring based Applications

The core Spring framework already reduces boilerplate code and provides a lot of helpful features for Java Applications.

However, Spring Boot takes this convenience to the next level by focusing specifically on reducing the effort required to set up and configure a Spring application.

->Auto-Configuration
->Standalone Applications

We need to manually set up the Spring application context in the main method.

In spring boot, @SpringBootApplication replaces manual configuration.(of RESTful API)

Automatic Component Scanning-> Detects and registers beans automatically.
Application Context-> Manages beans centrally for reuse.
Simplifies API Development->APIs can be built with minimal code.

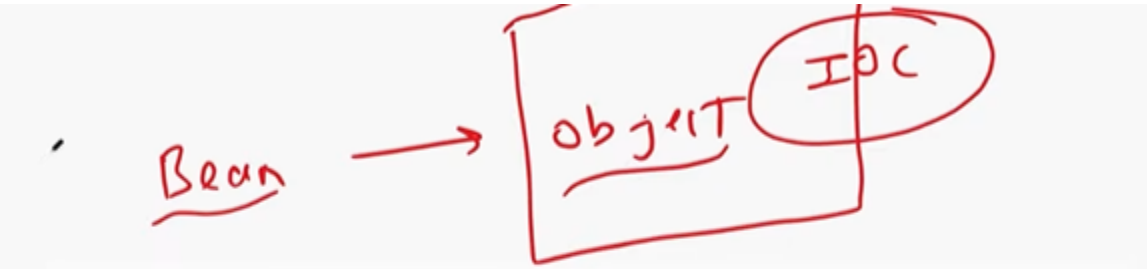
Beans are like the object just need to create once and can be called in code anywhere.

We make spring project using spring initialiser

#What spring does?
Car car=new Car() ; ->no need of this
Make the object for us ,we no longer need to create it manually,
Object creation ka kaam externalised krrahe ha->IOC container(jaha bahut object rakhe ha)->part of Spring boot.

Application context->way to achieve the IOC container.
Ab itni classes ma sa konsi classes IOC container apna andar rakhe ga ?
Jispr @Component annotation laga ha (It scan whole and jispr ya annota..)

What is bean?
Object in IOC container




```
@Component
public class MyComponent {
    // Class is automatically registered as a Spring bean.
}
```

@SpringBootApplication->poora spring boot project ma ek hi baar use hogi and vo main class ma lagayenga ,jaha main method hoga .



Scan the package in which it is only.

@RestController->it is also like a @Component,make bean+additional functionality ..Specialised version which handle HTTP requests.

@GetMapping->API banti ha ,end point create hojayega

@Configuration-> we can create bean inside that class using function

Creating REST APIs using Spring Boot->

Rest API ?
Representational State Transfer , Application Programming Interfaces

Intenet ,servers pr kuch chal raha ha ,ma ussa access kasie karunga?
Using Rest API

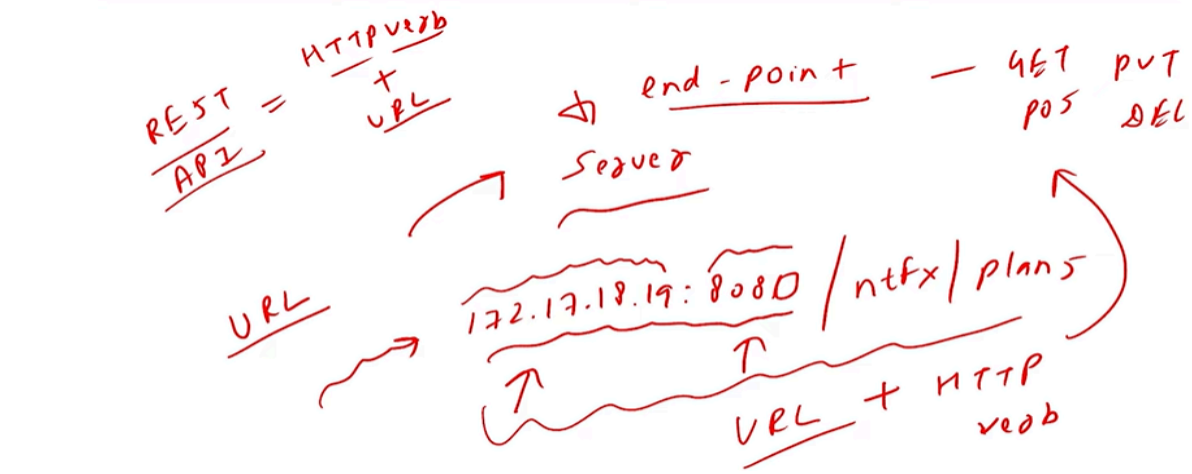
I am sending the request to the server,server will send us response ,
We access Server using RestAPI

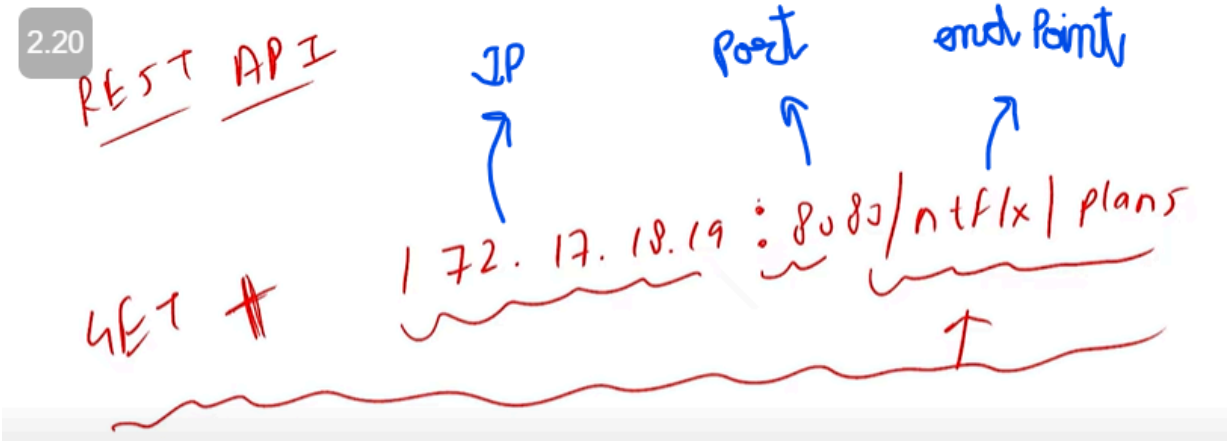
We access Server using URL,
Server expose one url,
We will not be able to access using url ->

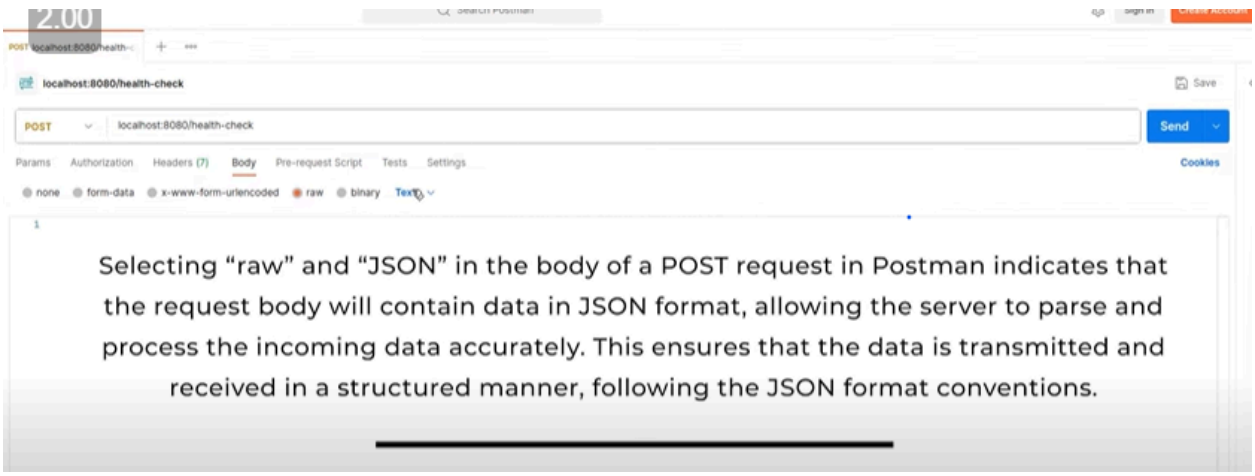
URL+HTTP verb

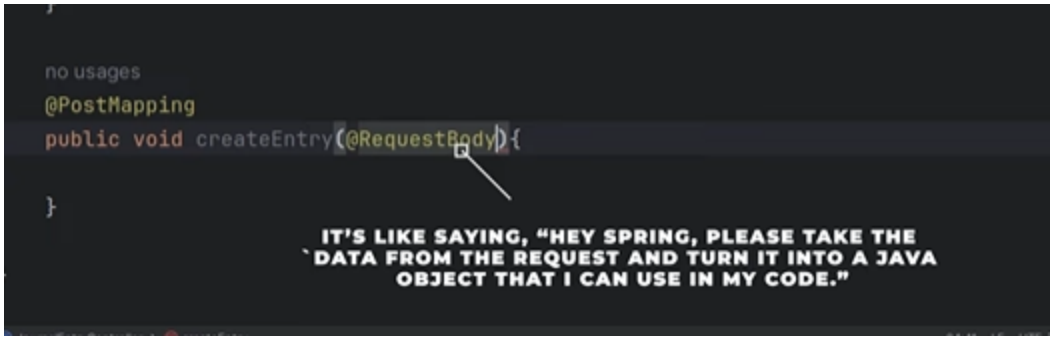
HTTP verb->

- GET -
- PUT-update
- POST-create
- DELETE

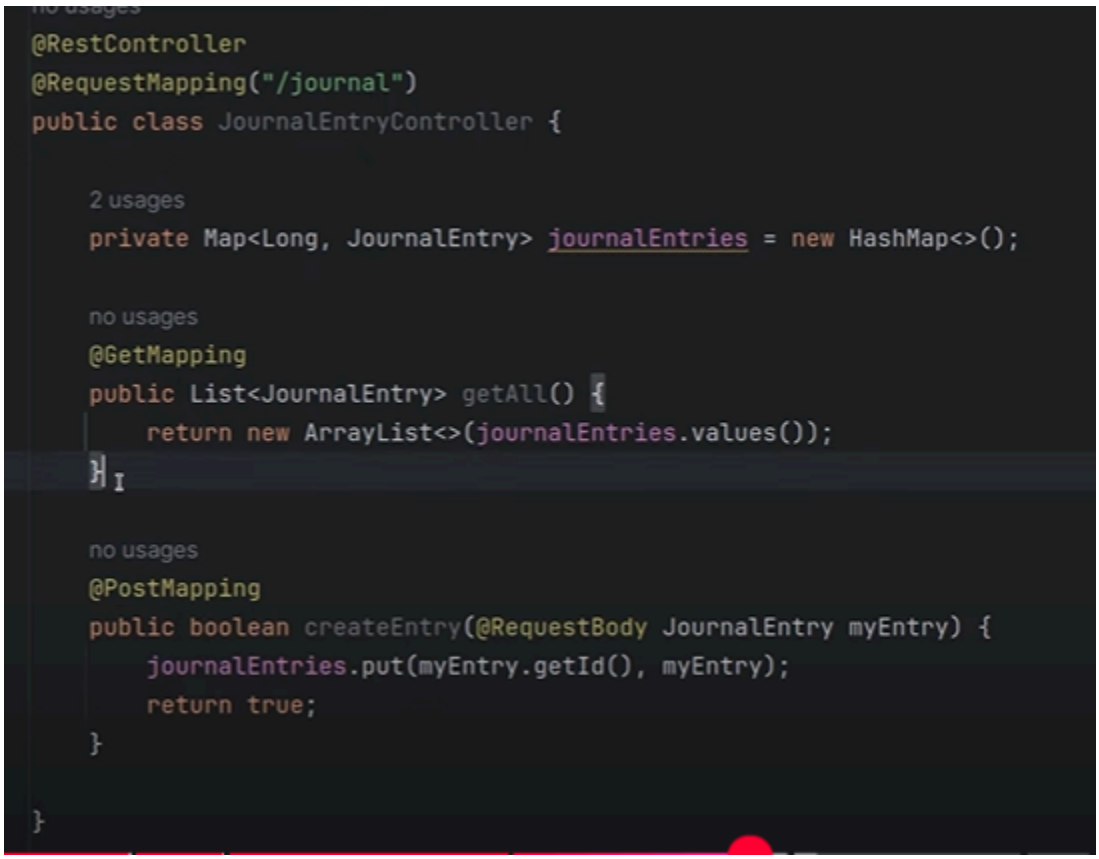

REST API = HTTP verb + URL
end-point
GET PUT POST DEL
Server
URL
172.17.18.19:8080/ntflx/plans
URL + HTTP verb


2.20
REST API
IP
Port
end Point
GET
172.17.18.19:8080/ntflx/plans


2.00
localhost:8080/health-check
POST
localhost:8080/health-check
Send
Params
Authorization
Headers (7)
Body
Pre-request Script
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
Text
Selecting “raw” and “JSON” in the body of a POST request in Postman indicates that the request body will contain data in JSON format, allowing the server to parse and process the incoming data accurately. This ensures that the data is transmitted and received in a structured manner, following the JSON format conventions.



(enable to take data from request coming from postman)



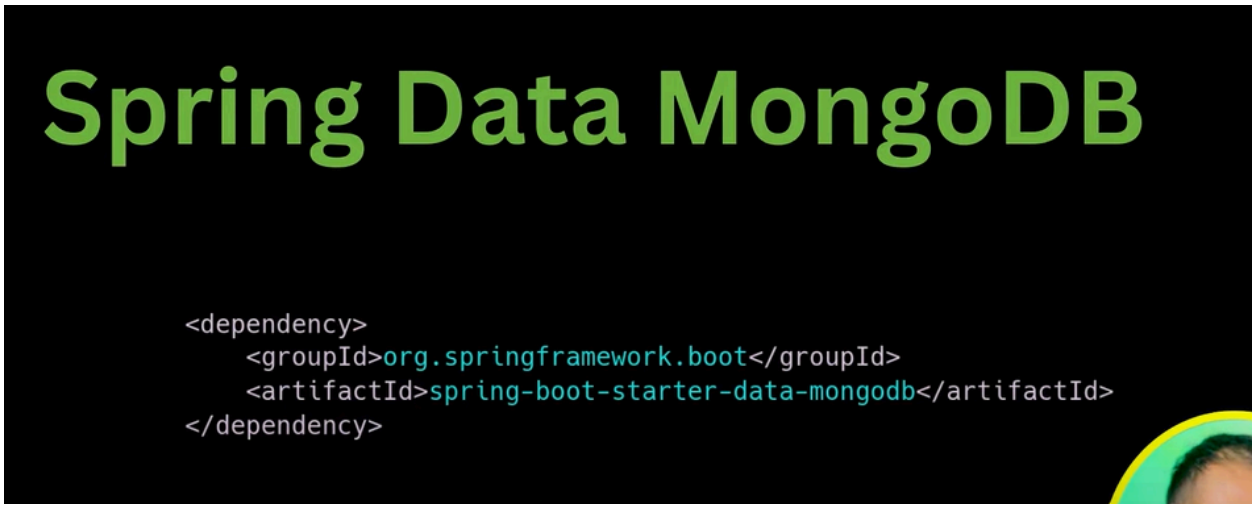
hence,here we created two APIs, (GET and POST)

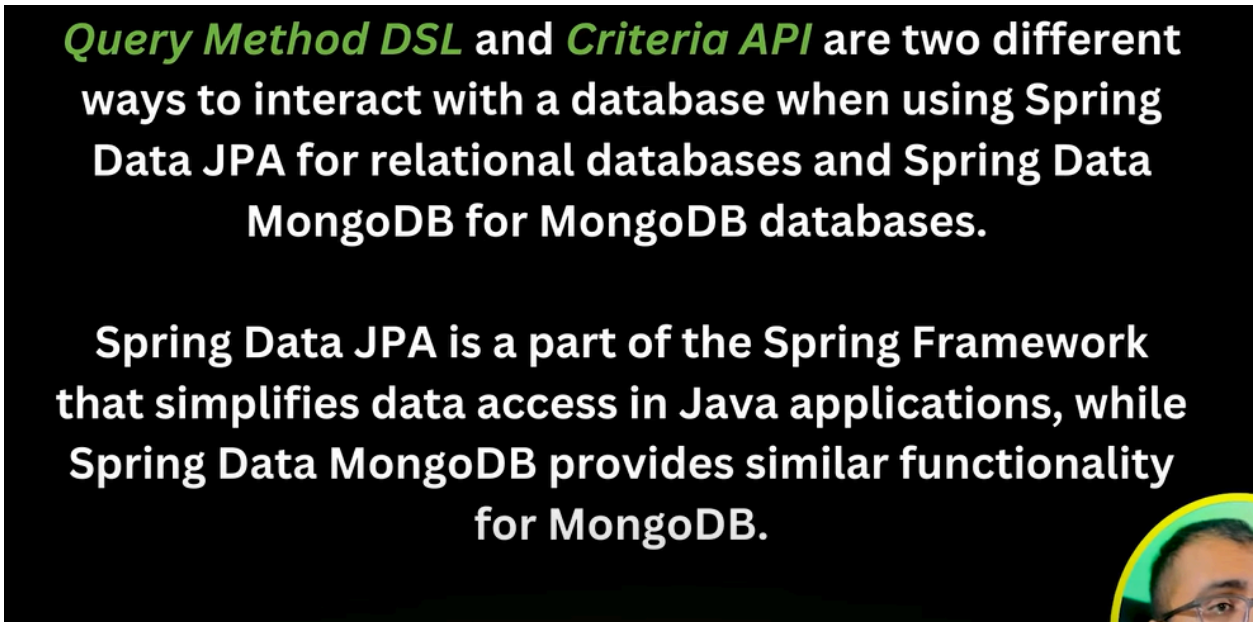
Open Mysql in command Prompt->

1) cd C:\Program Files\MySQL\MySQL Server 8.0\bin
2) mysql -u root -p
3) Write the password

Common Commands->

-Show databases;
-use user_db;
-show tables;
select* from cats;





Query Method DSL is a simple and convenient way to create queries based on method naming conventions, while the *Criteria API* offers a more dynamic and programmatic approach for building complex and custom queries



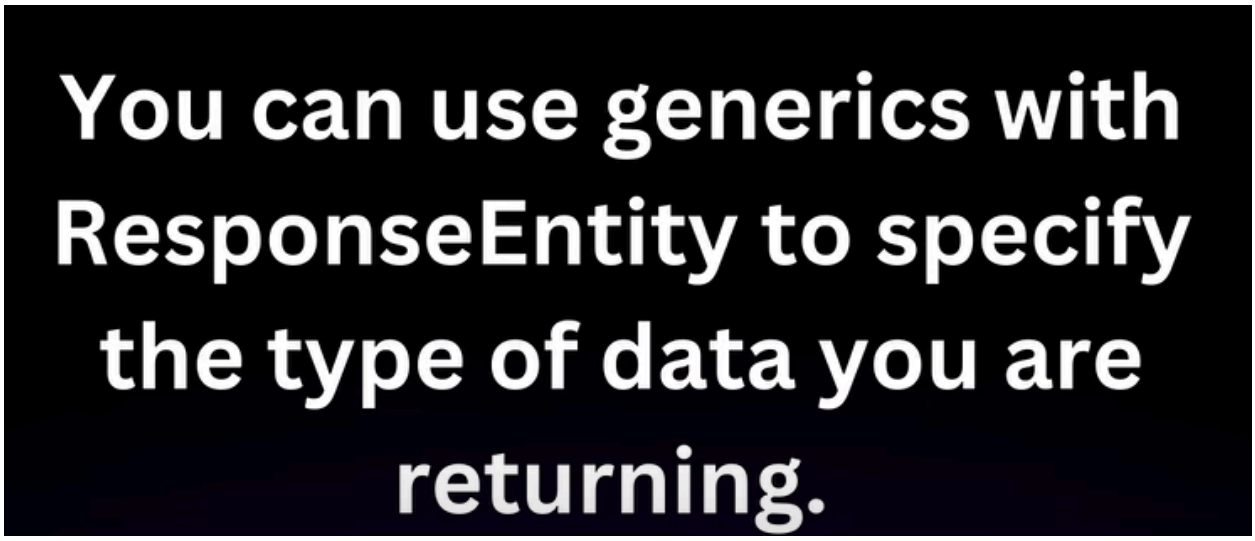
500 Internal Server Error: A generic error message indicating that something went wrong on the server, and the server could not handle the request.

502 Bad Gateway: The server acting as a gateway or proxy received an invalid response from an upstream server.

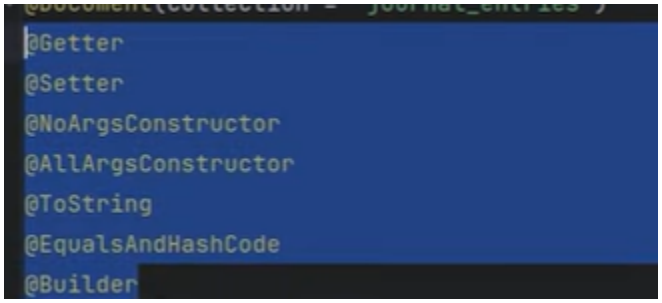
503 Service Unavailable: The server is currently unable to handle the request due to temporary overloading or maintenance.

ResponseEntity->

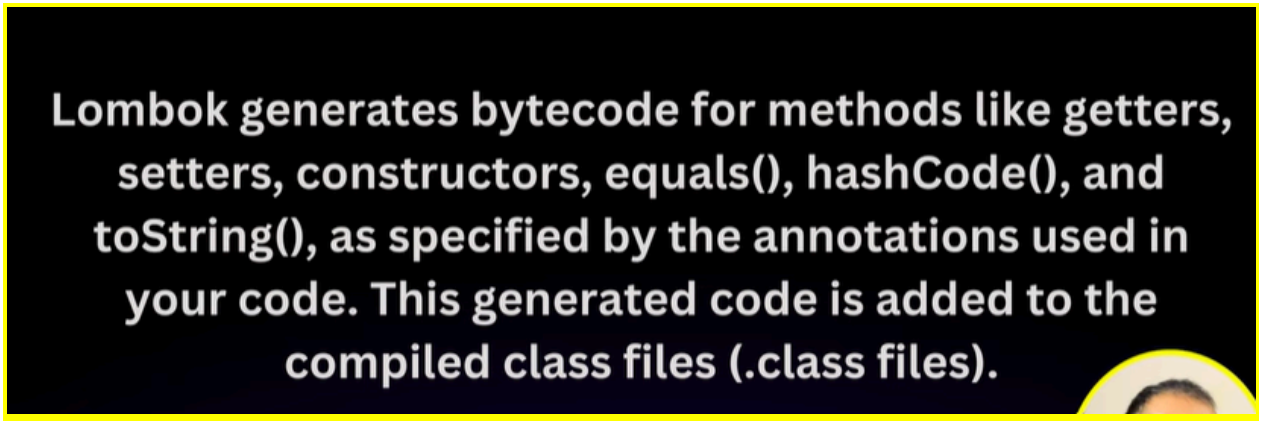
It provides methods for setting the response status, headers, and body. You can use it to return different types of data in your controller methods, such as JSON, XML, or even HTML.

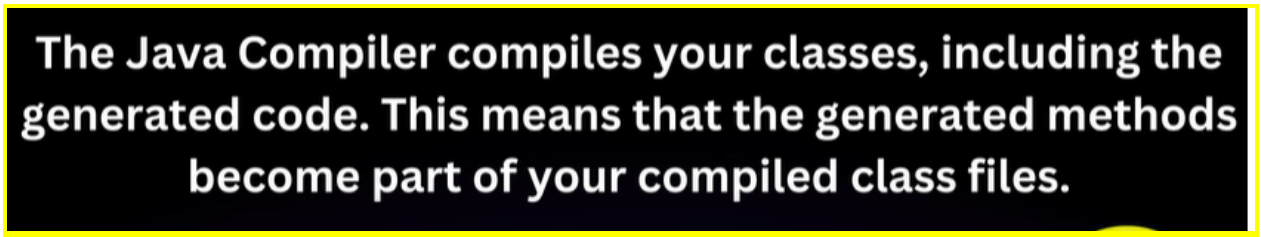


Lombok->



-> In sabhki jagah just use ine annotation-> @Data





When you run your application, the generated methods are available for use, just like any other methods in your classes.

Need an Index for Fast Searching?

- If you want **case-insensitive searches** on `userName`, use:
  sql
  Copy code
  ```
  CREATE INDEX idx_lower_username ON users
  (LOWER(userName));
  ```
- `: If unnecessary, you can drop the extra index using:`

  ```
  DROP INDEX idx_username ON users;
  ```

Spring Security

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Once the dependency is added, Spring Boot's auto-configuration feature will automatically apply security to the application.



The client sends an Authorization header

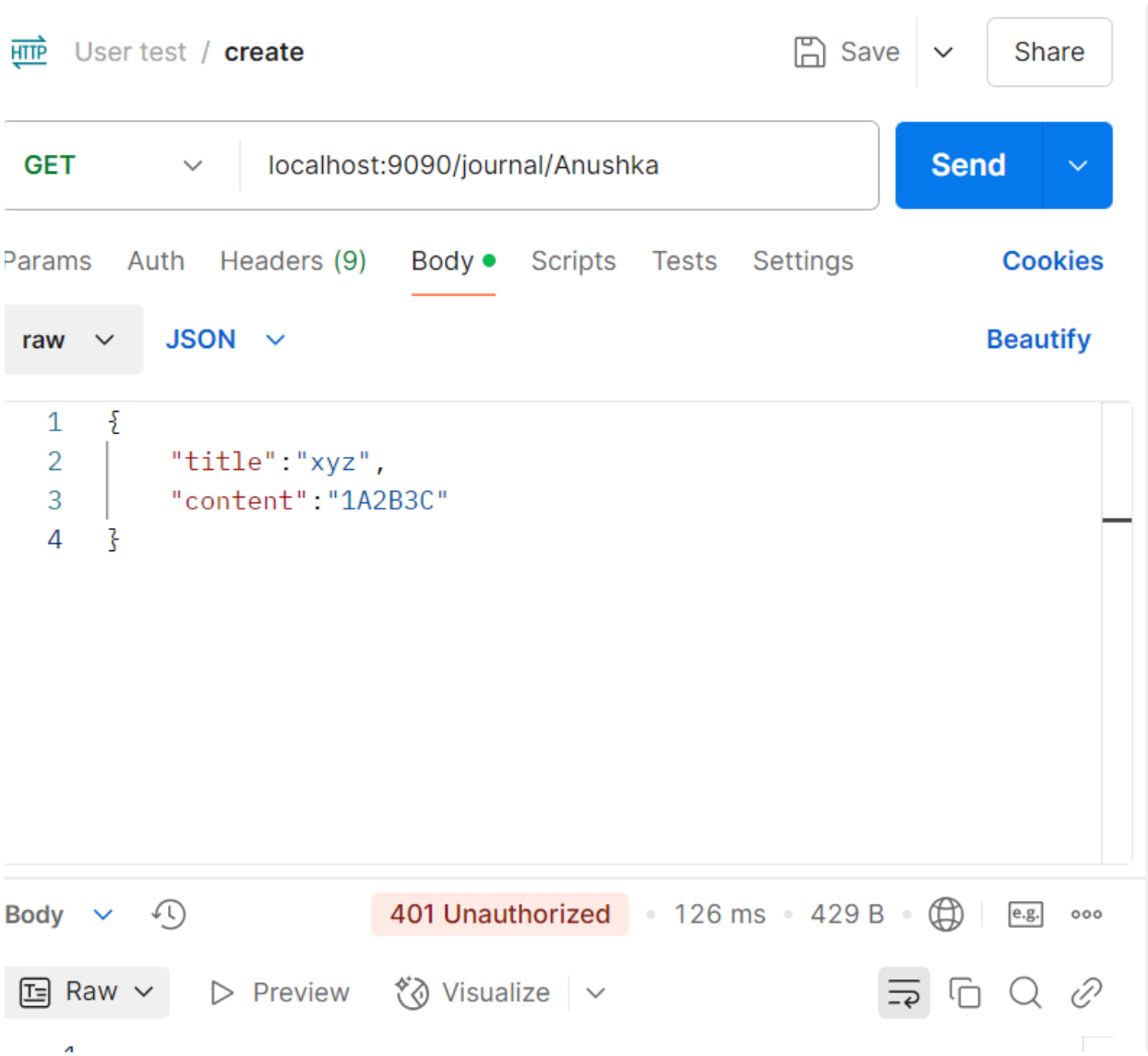
Authorization: Basic <encoded-string>

The server decodes the string, extracts the username and password, and verifies them. If they're correct, access is granted. Otherwise, an "Unauthorized" response is sent back.

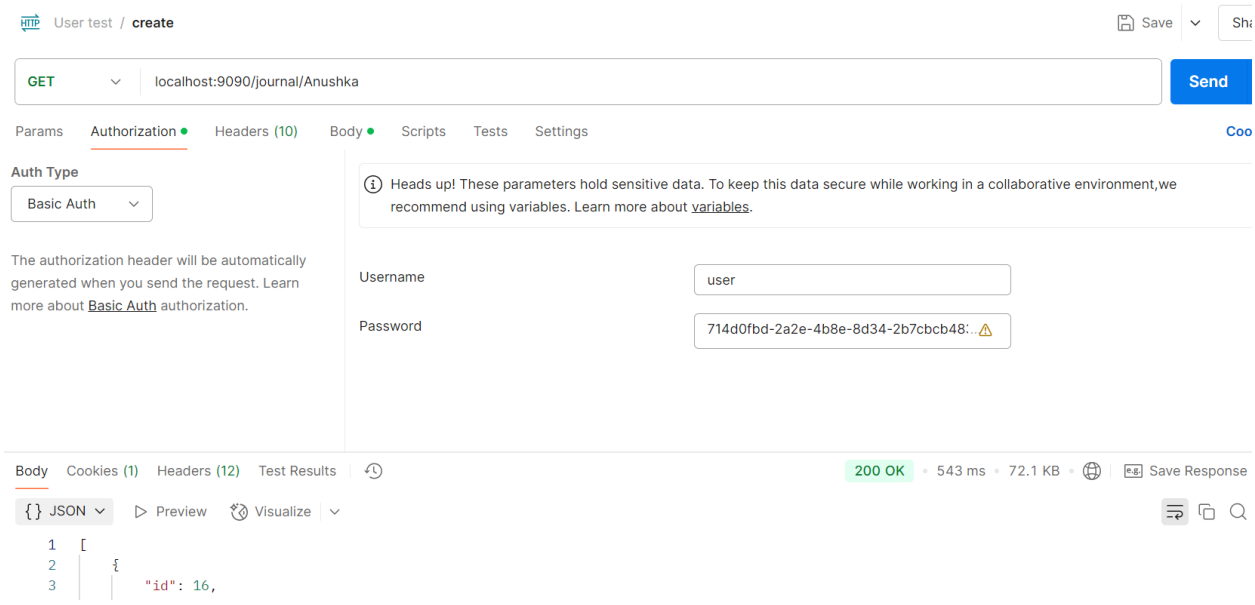


By default, all endpoints will be secured. Spring Security will generate a default user with a random password that is printed in the console logs on startup.





As soon as spring security dependency added,endpoints secured.now the User -> Unauthorized aagaya,it is asking ki username ,password use karo.



Customize Authentication

```java
@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            .authorizeRequests()
                .antMatchers("/hello").permitAll()
                .anyRequest().authenticated()
            .and()
            .formLogin();
    }
}
```

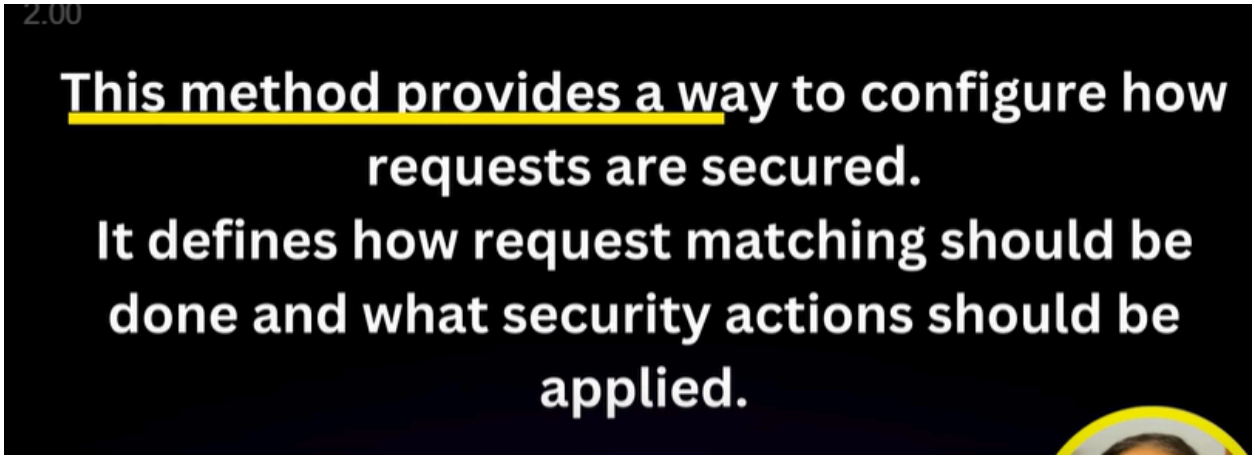
@EnableWebSecurity->



Human dependency bhi to add kari thi ussa bhi to spring ko ya hi pata lag raha tha ,what is special in this annotation?



Configure method->




```
HttpSecurity http
```

- Is sa hi we are able to apply filter(request- ki kisko authenticate krna ha, nhi krna)
- Instance
- Object

http.authorizeRequests(): This tells Spring Security to start authorizing the requests.

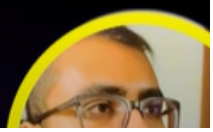
.antMatchers("/hello").permitAll(): This part specifies that HTTP requests matching the path /hello should be permitted (allowed) for all users, whether they are authenticated or not.

.anyRequest().authenticated(): This is a more general matcher that specifies any request (not already matched by previous matchers) should be authenticated, meaning users have to provide valid credentials to access these endpoints.

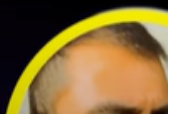
.and(): This is a method to join several configurations. It helps to continue the configuration from the root (HttpSecurity).

.formLogin(): This enables form-based authentication. By default, it will provide a form for the user to enter their username and password. If the user is not authenticated and they try to access a secured endpoint, they'll be redirected to the default login form.

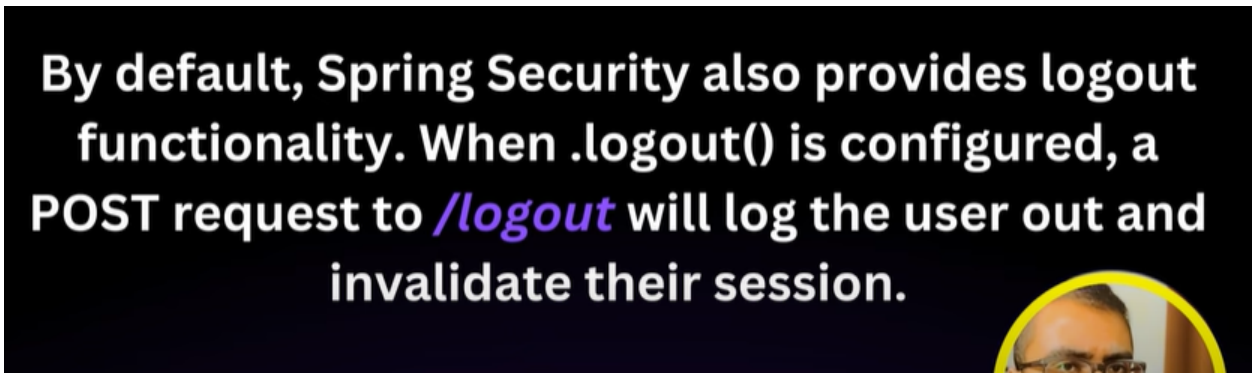
You can access /hello without any authentication. However, if you try to access another endpoint, you'll be redirected to a login form.

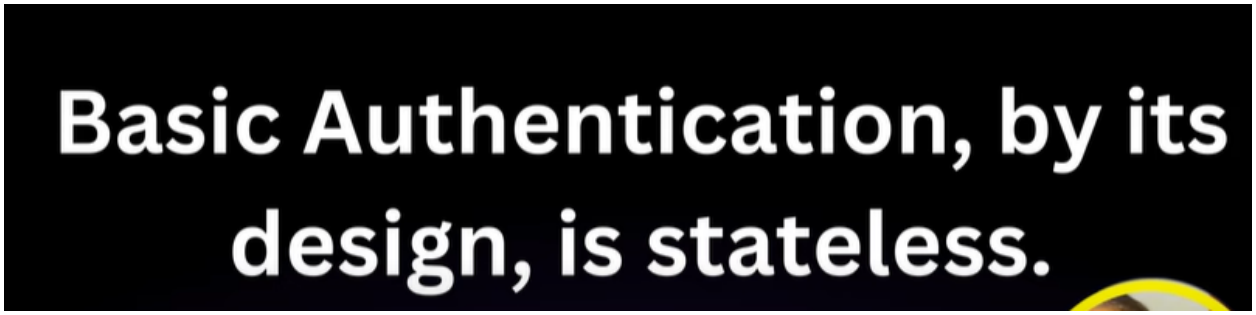


When we use the .formLogin() method in our security configuration without specifying .loginPage("/custom-path"), the default login page becomes active.

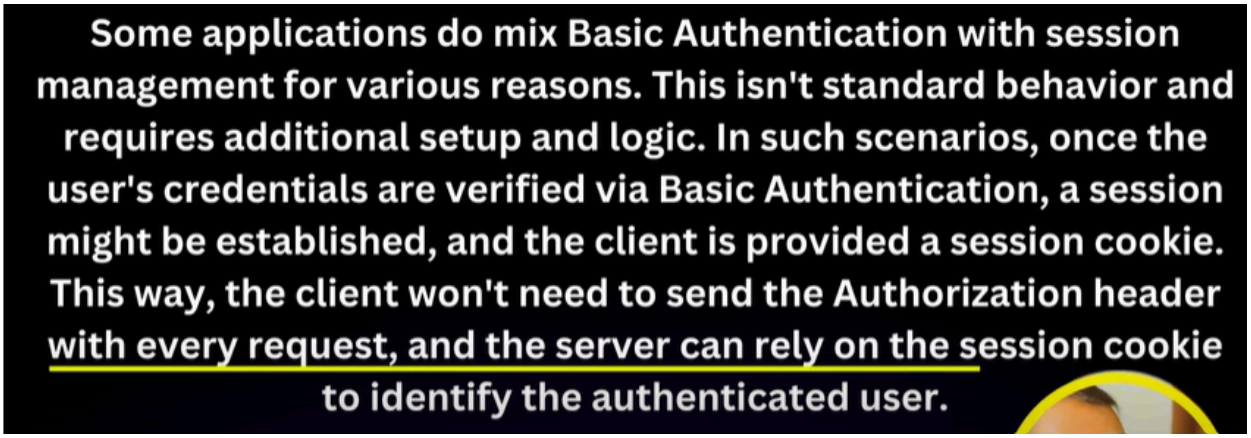








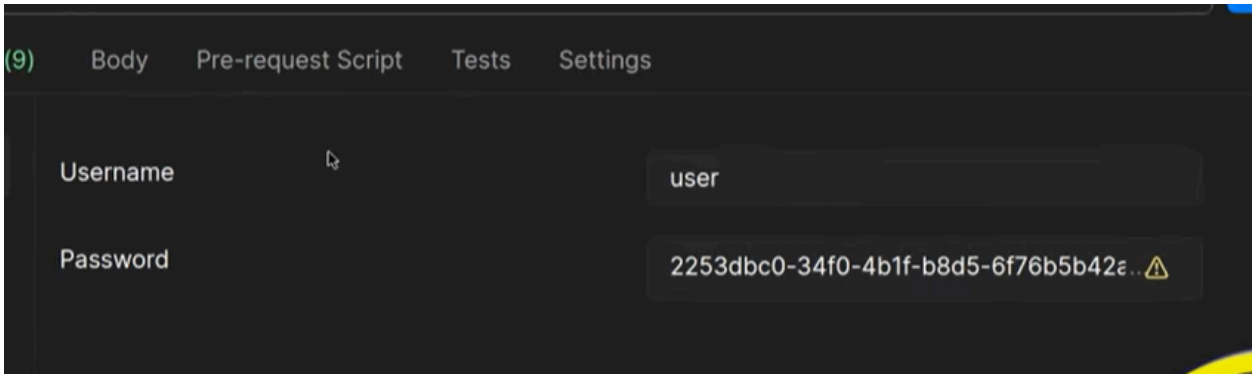
If stateless (state not stored)then what is the chakkar of logout????session cookie…..





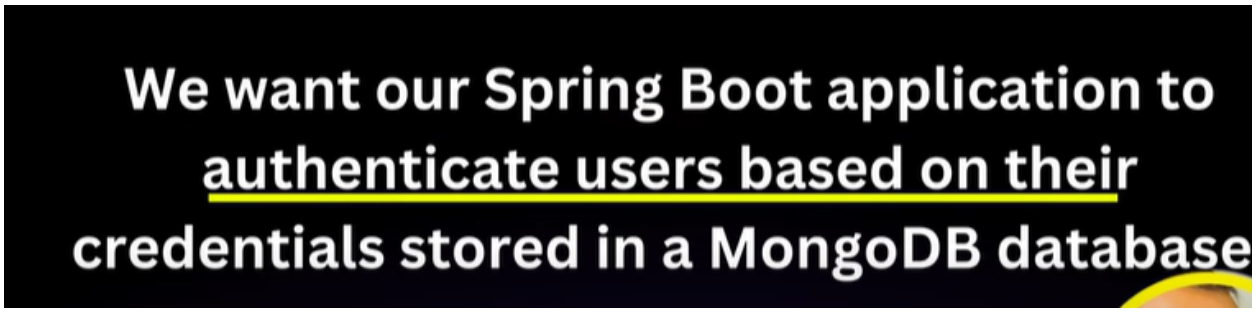
What is the meaning of session?
-Session means kuch cheejo ko yaad rakhna

Abhi tak to mera ek default user tha,but hamara users to db ma store ha na ,to unhi ko use krta ha.

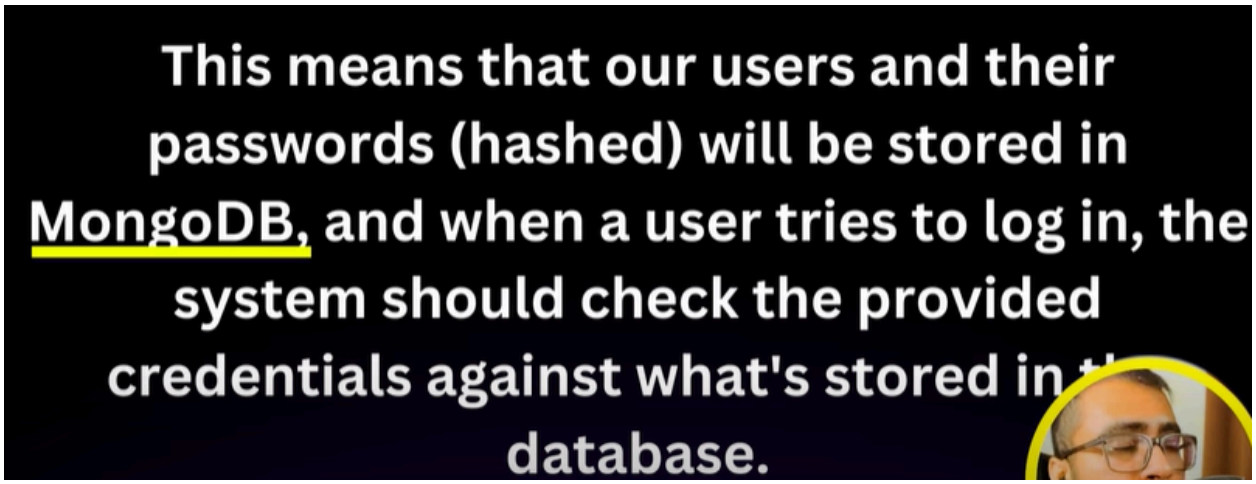


The user is common,so ya to sabhki journal entries dekh sakta ha ,but we didnt wanted that.[it is wrong]---kind of Admin role

We want particular user sirf apni entries dekh saka
And also our password is saved in plain text in database→ very wrong
We will encode this password and then will store in database



or sql Db



- [ ] A User entity to represent the user data model.
- [ ] A repository UserRepository to interact with MongoDB.
- [ ] UserDetailsService implementation to fetch user details.
- [ ] A configuration SecurityConfig to integrate everything with Spring Security.

1)

```java
@Document(collection = "users")
public class User {

    @Id
    private String id;
    private String username;
    private String password;
    private List<String> roles;

    // Getters, setters, and other methods...
}
```

(roles->related to authorization)

2)

```java
public interface UserRepository extends MongoRepository<User, String> {
    Optional<User> findByUsername(String username);
}
```

3)

```java
@Service
public class CustomUserDetailsService implements UserDetailsService {

    @Autowired
    private UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        User user = userRepository.findByUsername(username)
                .orElseThrow(() -> new UsernameNotFoundException("Username not found: " + username));

        return User.withUsername(user.getUsername())
                .password(user.getPassword())
                .roles(user.getRoles().toArray(new String[0]))
                .build();
    }
}
```

4)

```java
@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {

    @Autowired
    private CustomUserDetailsService userDetailsService;

    @Override
    protected void configure(AuthenticationManagerBuilder auth) throws Exception {
        auth.userDetailsService(userDetailsService).passwordEncoder(passwordEncoder());
    }

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            .authorizeRequests()
                .antMatchers("/hello").permitAll()
                .anyRequest().authenticated()
            .and()
            .formLogin();
    }

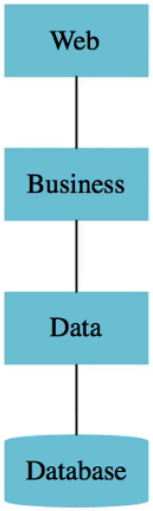
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

}
```

Introduction to Unit Testing with JUnit

In 28 Minutes

- Large applications can have **1000s of code files** and **millions of lines of code**
- **Testing**: Check app behavior against expected behavior
  - **Option 1**: Deploy the complete application and test
    - This is called **System Testing** or **Integration Testing**
  - **Option 2**: Test specific units of application code independently
    - Examples: A specific method or group of methods
    - This is called **Unit Testing**
    - **Advantages of Unit Testing**
      - Finds bug early (run under CI)
      - Easy to fix bugs
      - Reduces costs in the long run
    - **Most Popular Java Frameworks**: JUnit and Mockito
  - **Recommended**: Option 1 + Option 2



`assertEquals` is a JUnit assertion method used to check whether two values are equal in a test case. If they are not equal, the test fails.

Introduction to Mocking with Mockito

- Great unit tests => Easier Maintenance
- **However**, writing great units is **NOT easy**
  - Applications have multiple layers
    - Each class has multiple dependencies
- How do you write unit test for classes with multiple dependencies?
  - **Option 1**: Stubs
  - **Option 2**: Mocks

Example->Stubs:

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }
}

class CalculatorStub extends Calculator {
    @Override
    int add(int a, int b) {
        return 10; // Always return 10, no matter the input
    }
}
```

Here, `CalculatorStub` overrides `add()` to always return `10`, ignoring actual inputs.

Mock

A mock is a **fully controlled** fake object that can simulate method calls, verify interactions, and return different responses based on inputs.

```
import static org.mockito.Mockito.*;
import org.junit.jupiter.api.Test;


class TestExample {
    @Test
    void testWithMock() {
        // Create a mock object of Calculator
        Calculator calculator = mock(Calculator.class);

        // Define behavior: when add(2,3) is called, return 10
        when(calculator.add(2, 3)).thenReturn(10);

        // Test
        System.out.println(calculator.add(2, 3)); // Output: 10
    }
}
```

2. Here, `mock(Calculator.class)` creates a **mock** calculator, and we tell it:
    - **"If someone calls add(2, 3), return 10"**
    - The real method is **never called**; Mockito controls the output.

For implementing unit testing->

In pom.xml->

```
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-test</artifactId>
    <scope>test</scope>
</dependency>

<dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <scope>test</scope>
</dependency>

</dependency>

<dependency>
    <groupId>org.mockito</groupId>
    <artifactId>mockito-core</artifactId>
    <scope>test</scope>
</dependency>
```

Defining context_configuration using class->

```
package com.in28minutes.spring.basics.springin5steps.basic;

import static org.junit.Assert.assertEquals;

//Load the context
@RunWith(SpringRunner.class)
@ContextConfiguration(classes = SpringIn5StepsBasicApplication.class)
public class BinarySearchXMLConfigurationTest {

    // Get this bean from the context
    @Autowired
    BinarySearchImpl binarySearch;

    @Test
    public void testBasicScenario() {
```

```
  // Get this bean from the context
  @Autowired
  BinarySearchImpl binarySearch;

  @Test
  public void testBasicScenario() {

      // call method on binarySearch
      int actualResult = binarySearch.binarySearch(new int[] {}, 5);

      // check if the value is correct
      assertEquals(3, actualResult);

  }
```

XML->
In place of classes we will use locations

```java
//Load the context
@RunWith(SpringRunner.class)
@ContextConfiguration(locations="/applicationContext.xml")
public class BinarySearchXMLConfigurationTest {

    // Get this bean from the context
    @Autowired
    BinarySearchImpl binarySearch;

    @Test
    public void testBasicScenario() {

        // call method on binarySearch
        int actualResult = binarySearch.binarySearch(new int[] {}, 5);
```

Using MOCK->

```java
@RunWith(MockitoJUnitRunner.class)
public class SomeCdiBusinessTest {

    //Inject Mock
    @InjectMocks
    SomeCdiBusiness business;

    //Create Mock
    @Mock
    SomeCdiDao daoMock;

    @Test
    public void testBasicScenario() {
        Mockito.when(daoMock.getData()).thenReturn(new int[]{2,4});
        //when daoMock.getData() method is called return int[]{2,4}

        int actualResult = business.findGreatest();
```

World Before Spring Boot!

- Setting up Spring Projects **before Spring Boot was NOT easy**!
  - 1: Dependency Management (**pom.xml**)
  - 2: Define Web App Configuration (**web.xml**)
  - 3: Manage Spring Beans (**context.xml**)
  - 4: Implement Non Functional Requirements (NFRs)
- AND repeat this for every new project!
- Typically takes a **few days** to setup for each project (and countless hours to maintain)

World Before Spring Boot - 1 - Dependency Management

```
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-webmvc</artifactId>
    <version>5.2.2.RELEASE</version>
</dependency>
<dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
    <version>2.13.3</version>
</dependency>
<dependency>
    <groupId>log4j</groupId>
    <artifactId>log4j</artifactId>
    <version>1.2.17</version>
</dependency>
```

- Manage frameworks and versions
  - **REST API** - Spring framework, Spring MVC framework, JSON binding framework, ..
  - **Unit Tests** - Spring Test, Mockito, JUnit, ...

World Before Spring Boot - 2 - web.xml

```
<servlet>
    <servlet-name>dispatcher</servlet-name>
    <servlet-class>
        org.springframework.web.servlet.DispatcherServlet
    </servlet-class>
    <init-param>
        <param-name>contextConfigLocation</param-name>
        <param-value>/WEB-INF/todo-servlet.xml</param-value>
    </init-param>
    <load-on-startup>1</load-on-startup>
</servlet>

<servlet-mapping>
    <servlet-name>dispatcher</servlet-name>
    <url-pattern>/*</url-pattern>
</servlet-mapping>
```

- **Example**: Configure **DispatcherServlet** for Spring MVC

World Before Spring Boot - 3 - Spring Configuration

```
<context:component-scan base-package="com.in28minutes" />

<bean
    class="org.springframework.web.servlet.view.InternalResourceViewResolver">
    <property name="prefix">
        <value>/WEB-INF/views/</value>
    </property>
    <property name="suffix">
        <value>.jsp</value>
    </property>
</bean>
```

- Define your **Spring Configuration**
  - Component Scan
  - View Resolver

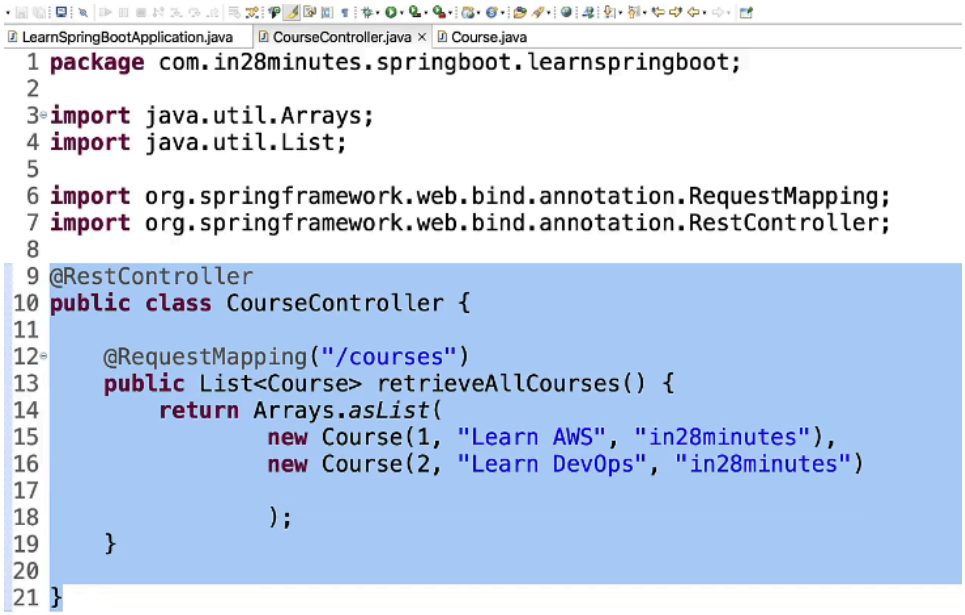
World Before Spring Boot - 4 - NFRs

```
<plugin>
    <groupId>org.apache.tomcat.maven</groupId>
    <artifactId>tomcat7-maven-plugin</artifactId>
    <version>2.2</version>
    <configuration>
        <path>/</path>
        <contextReloadable>true</contextReloadable>
    </configuration>
</plugin>

<dependency>
    <groupId>log4j</groupId>
    <artifactId>log4j</artifactId>
    <version>1.2.17</version>
</dependency>
```

- Logging
- Error Handling
- Monitoring

First spring boot project->



LearnSpringBootApplication.java *CourseController.java *Course.java ×

```java
package com.in28minutes.springboot.learnspringboot;

public class Course {
    private long id;
    private String name;
    private String author;

    // Constructor
    // Getters

    public long getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public String getAuthor() {
        return author;
    }

    @Override
    public String toString() {
        return "Course [id=" + id + ", name=" + name + ", author=" + author + "]";
    }
```

localhost:8080/courses


```
[
  {
    "id": 1,
    "name": "Learn AWS",
    "author": "in28minutes"
  },
  {
    "id": 2,
    "name": "Learn DevOps",
    "author": "in28minutes"
  }
]
```


Exploring Spring Boot Starter Projects

- I need a lot of frameworks to build application features:
  - **Build a REST API**: I need Spring, Spring MVC, Tomcat, JSON conversion...
  - **Write Unit Tests**: I need Spring Test, JUnit, Mockito, ...
- How can I group them and make it easy to build applications?
  - **Starters**: Convenient **dependency descriptors** for diff. features
- **Spring Boot** provides variety of starter projects:
  - **Web Application & REST API** - Spring Boot Starter Web (spring-webmvc, spring-web, spring-boot-starter-tomcat, spring-boot-starter-json)
  - **Unit Tests** - Spring Boot Starter Test
  - **Talk to database using JPA** - Spring Boot Starter Data JPA
  - **Talk to database using JDBC** - Spring Boot Starter JDBC
  - **Secure your web application or REST API** - Spring Boot Starter Security

LearnSpringBootApplication.java CourseController.java Course.java learn-spring

```
1 logging.level.org.springframework=debug
2
```

logs-

Exploring Spring Boot Auto Configuration

- I need **lot of configuration** to build Spring app:
  - Component Scan, DispatcherServlet, Data Sources, JSON Conversion, ...
- How can I simplify this?
  - **Auto Configuration: Automated configuration** for your app
    - **Decided** based on:
      - Which frameworks are in the Class Path?
      - What is the existing configuration (Annotations etc)?
- **Example**: Spring Boot Starter Web
  - Dispatcher Servlet (`DispatcherServletAutoConfiguration`)
  - Embedded Servlet Container - Tomcat is the default (`EmbeddedWebServerFactoryCustomizerAutoConfiguration`)
  - Default Error Pages (`ErrorMvcAutoConfiguration`)
  - Bean<->JSON (`JacksonHttpMessageConvertersConfiguration`)

Build Faster with Spring Boot DevTools

- Increase developer productivity
- Why do you need to restart the server **manually** for every code change?
- **Remember**: For pom.xml dependency changes, you will need to restart server **manually**



only ctrl+s

Multiple logging levels are supported

application.propert | *notes.txt | application-dev.pro | application-prod.pr | Currency

```
1
2 dev
3 ~~~~
4 logging.level.org.springframework=trace
5
6 prod
7 ~~~~
8 logging.level.org.springframework=info
9
10
11 trace
12 debug
13 info
14 warning
15 error
16 off
```

Monitor Applications using Spring Boot Actuator

In 28 Minutes

- Monitor and manage your application in your production
- Provides a number of endpoints:
  - **beans** - Complete list of Spring beans in your app
  - **health** - Application health information
  - **metrics** - Application metrics
  - **mappings** - Details around Request Mappings



Understanding Spring Boot vs Spring MVC vs Spring

- **Spring Boot vs Spring MVC vs Spring**: What's in it?
  - **Spring Framework**: Dependency Injection
    - @Component, @Autowired, Component Scan etc..
    - Just Dependency Injection is NOT sufficient (You need other frameworks to build apps)
      - **Spring Modules and Spring Projects**: Extend Spring Eco System
        - Provide good integration with other frameworks (Hibernate/JPA, JUnit & Mockito for Unit Testing)
  - **Spring MVC** (Spring Module): Simplify building web apps and REST API
    - Building web applications with Struts was very complex
    - @Controller, @RestController, @RequestMapping("/courses")
  - **Spring Boot** (Spring Project): Build **PRODUCTION-READY** apps **QUICKLY**
    - **Starter Projects** - Make it easy to build variety of applications
    - **Auto configuration** - Eliminate configuration to setup Spring, Spring MVC and other frameworks!
    - Enable non functional requirements (NFRs):
      - **Actuator**: Enables Advanced Monitoring of applications
      - **Embedded Server**: No need for separate application servers!
      - Logging and Error Handling
      - Profiles and ConfigurationProperties

Spring JDBC,JPA ,H2

1) H2 database name is randomly generated each time you restart the server.

2) Data Loading from data.sql needs additional configuration

RECOMMENDED:

Use this configuration in `application.properties`

```
spring.datasource.url=jdbc:h2:mem:testdb;NON_KEYWORDS=USER
spring.h2.console.enabled=true    //enable the console
spring.data.jpa.repositories.bootstrap-mode=default
spring.jpa.defer-datasource-initialization=true
```

#Use spring JDBC to execute our query and print the result(git->step6)

/src/main/java/com/in28minutes/database/databasedemo/entity/Person.java

```
package com.in28minutes.database.databasedemo.entity;

import java.util.Date;

public class Person {
        private int id;
        private String name;
        private String location;
        private Date birthDate;

        public Person() {

        }

        public Person(int id, String name, String location, Date birthDate) {
                super();
                this.id = id;
                this.name = name;
                this.location = location;
                this.birthDate = birthDate;
        }

        public int getId() {
                return id;
        }
```

/src/main/java/com/in28minutes/database/databasedemo/jdbc/PersonJbdcDao.java

```
package com.in28minutes.database.databasedemo.jdbc;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.BeanPropertyRowMapper;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Repository;

import com.in28minutes.database.databasedemo.entity.Person;

@Repository
public class PersonJbdcDao {
        @Autowired
        JdbcTemplate jdbcTemplate;

        public List<Person> findAll() {
                return jdbcTemplate.query("select * from person",
                                new BeanPropertyRowMapper<Person>(Person.class));
        }
}
```

/src/main/java/com/in28minutes/database/databasedemo/DatabaseDemoApplication.java

```
package com.in28minutes.database.databasedemo;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

import com.in28minutes.database.databasedemo.jdbc.PersonJbdcDao;

@SpringBootApplication
public class DatabaseDemoApplication implements CommandLineRunner {

	private Logger logger = LoggerFactory.getLogger(this.getClass());

	@Autowired
	PersonJbdcDao dao;

	public static void main(String[] args) {
		SpringApplication.run(DatabaseDemoApplication.class, args);
	}

	@Override
	public void run(String... args) throws Exception {
		logger.info("All users -> {}", dao.findAll());
	}
}
```

/src/main/resources/data.sql

```
create table person
(
  id integer not null,
  name varchar(255) not null,
  location varchar(255),
  birth_date timestamp,
  primary key(id)
);

INSERT INTO PERSON (ID, NAME, LOCATION, BIRTH_DATE )
VALUES(10001,  'Ranga', 'Hyderabad',sysdate());
INSERT INTO PERSON (ID, NAME, LOCATION, BIRTH_DATE )
VALUES(10002,  'James', 'New York',sysdate());
INSERT INTO PERSON (ID, NAME, LOCATION, BIRTH_DATE )
VALUES(10003,  'Pieter', 'Amsterdam',sysdate());
```

##queryForObject method accepts the list of parameters

```
new BeanPropertyRowMapper<Person>(Person.class));
```

```
package webapp;

import java.io.IOException;
import java.io.PrintWriter;

import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

/*
* Browser sends Http Request to Web Server
*
* Code in Web Server => Input:HttpRequest, Output: HttpResponse
* JEE with Servlets
*
* Web Server responds with Http Response
*/

//Java Platform, Enterprise Edition (Java EE) JEE6

//Servlet is a Java programming language class
//used to extend the capabilities of servers
//that host applications accessed by means of
//a request-response programming model.

//1. extends javax.servlet.http.HttpServlet
//2. @WebServlet(urlPatterns = "/login.do")
//3. doGet(HttpServletRequest request, HttpServletResponse response)
//4. How is the response created?

@WebServlet(urlPatterns = "/login.do")
public class LoginServlet extends HttpServlet {
```

```java
   @Override
       protected  void  doGet(HttpServletRequest  request,  HttpServletResponse
response) throws IOException {
      PrintWriter out = response.getWriter();
      out.println("<html>");
      out.println("<head>");
      out.println("<title>Yahoo!!!!!!!!</title>");
      out.println("</head>");
      out.println("<body>");
      out.println("My First Servlet");
      out.println("</body>");
      out.println("</html>");

   }

}
```

```
http://localhost:8080/spring-mvc/login

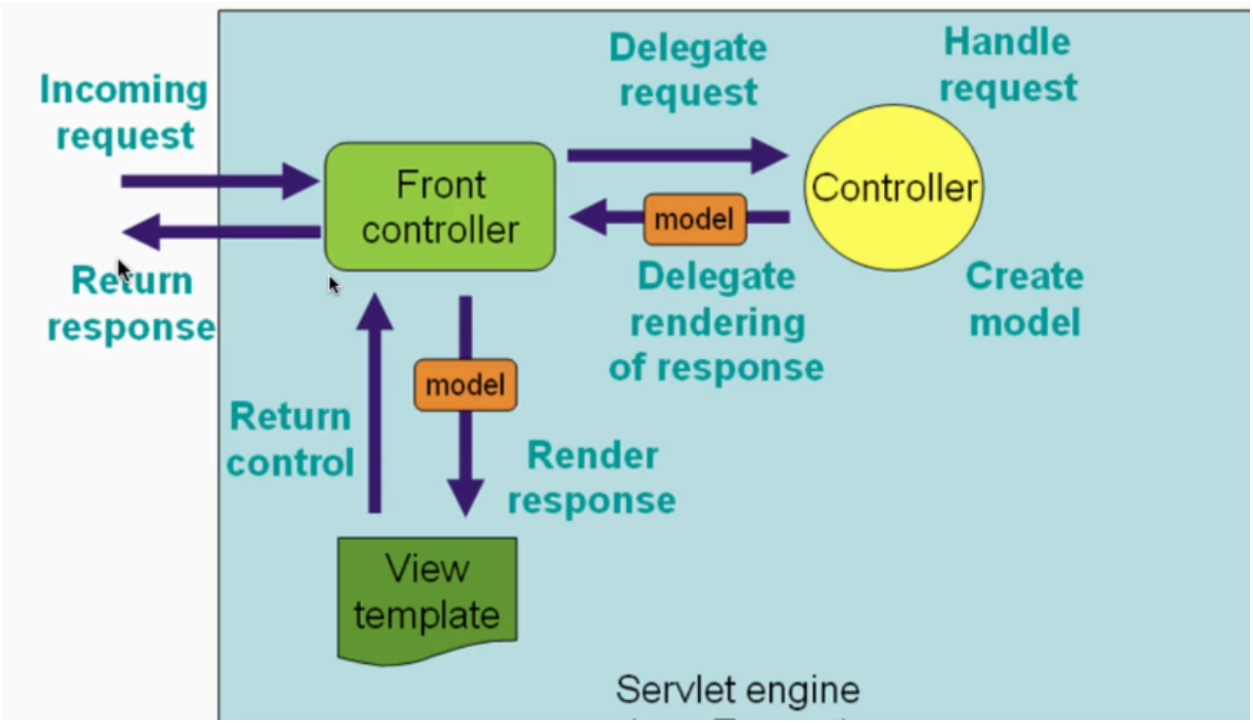
spring-mvc
DispatcherServlet -> Front Controller

/login -> LoginController (Handler)
Hello World

View Resolver
login -> /WEB-INF/views/login.jsp

login.jsp -> View

/WEB-INF/views/login.jsp
```



Spring MVC Request Flow

- DispatcherServlet receives HTTP Request.
- DispatcherServlet identifies the right Controller based on the URL.
- Controller executes Business Logic.
- Controller returns a) Model b) View Name Back to DispatcherServlet.
- DispatcherServlet identifies the correct view (ViewResolver).
- DispatcherServlet makes the model available to view and executes it.
- DispatcherServlet returns HTTP Response Back.
- Flow : http://docs.spring.io/spring-framework/docs/2.0.8/reference/images/mvc.png

PROJECT->AUTHENTICATION

JWT FILTER

- **Purpose**: The `JwtFilter` ensures that only authenticated users can access protected endpoints by validating JWTs.
- **Key Steps**:
    1. Extract the JWT from the `Authorization` header.

2. Validate the JWT and extract the username.
3. Load user details and set the authentication in the SecurityContext.
4. Continue the filter chain.

This filter is a critical part of securing a Spring Boot

End-to-End Data Flow: MySQL → Redis → Application

Let's break it down step by step.

1️⃣ How Data is Stored in MySQL?

When we save a user object in MySQL, it is stored in **relational table format**.

Example: `users` Table in MySQL

id	name	role
1	Alice	ADMIN
2	Bob	USER

Java Entity Class (`UserInfo.java`)

```
@Entity
@Table(name = "users")
public class UserInfo {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String role;
}
```

- The **Java object is converted into a row** in the MySQL table.
- **Stored format:** Relational table with columns.

2️⃣ Fetching Data from MySQL in Java (JPA Repository)

When we query MySQL using **Spring Data JPA**, it retrieves the data as a **Java Object**.

java

CopyEdit

```
Optional<UserInfo> userInfo = userInfoRepository.findById(1L);
```

This fetches **row data from MySQL** and converts it into a **Java object** (`UserInfo`).

Example:

java

CopyEdit

```
UserInfo{id=1, name="Alice", role="ADMIN"}
```

3️⃣ Storing the Data in Redis (Serialization)

Why Serialization?

Redis only understands **binary or text-based formats** (e.g., **JSON, String, or ByteArray**).
So, before storing the `UserInfo` object, we **convert (serialize)** it into **JSON format**.

Storing in Redis Using `RedisTemplate`

java

CopyEdit

```
redisTemplate.opsForValue().set("user:1", userInfo);
```

Behind the scenes:

1. `userInfo` Java object → **Converted to JSON**
2. JSON is **stored in Redis** as a key-value pair.

How Data is Stored in Redis?

Redis Key	Redis Value (JSON)
`user:1`	`{"id":1,"name":"Alice","role":"ADMIN"}`

- **Key**: `"user:1"`
- **Value**: **Serialized JSON** version of `UserInfo`

4️⃣ Fetching Data from Redis (Deserialization)

Now, when we request the user again, we **first check Redis**.

java

CopyEdit

```
UserInfo cachedUser = (UserInfo) redisTemplate.opsForValue().get("user:1");
```

Redis **returns the JSON**:
json
CopyEdit
```
{"id":1,"name":"Alice","role":"ADMIN"}
```

-
- We **convert (deserialize) JSON back into a Java object**.

Now, the Java object is:

java

CopyEdit

```
UserInfo{id=1, name="Alice", role="ADMIN"}
```

This object is **returned to the application**.

5️⃣ Final Flow: End-to-End Process

Step-by-Step

1️⃣ Save user in MySQL

- Java object → **Stored in MySQL as a table row**.

  2️⃣ **Fetch user from MySQL**
- MySQL row → **Converted into a Java object**.

  3️⃣ **Store user in Redis**
- Java object → **Serialized to JSON & stored in Redis**.

  4️⃣ **Fetch user from Redis**
- JSON → **Deserialized back into a Java object**.

🔄 Full Data Conversion Cycle

Stage	Data Format
MySQL	Relational Table (Rows)

Fetch in Java	Java Object (`UserInfo`)
Store in Redis	JSON (Serialized Data)
Fetch from Redis	JSON → Java Object (Deserialized)

Example Code for Storing and Retrieving Data

Service Layer (`UserService.java`)

java

CopyEdit

```
@Service
public class UserService {

    @Autowired
    private RedisTemplate<String, Object> redisTemplate;

    @Autowired
    private UserRepository userRepository;

    public UserInfo getUserById(Long userId) {
        // Step 1: Check Redis cache
        UserInfo cachedUser = (UserInfo) redisTemplate.opsForValue().get("user:" + userId);

        if (cachedUser != null) {
```

```
            System.out.println("Fetching from Redis Cache");
            return cachedUser; // Return cached user
        }

        // Step 2: If not in Redis, fetch from MySQL
        UserInfo userFromDB = userRepository.findById(userId).orElse(null);

        if (userFromDB != null) {
            // Step 3: Store in Redis
            redisTemplate.opsForValue().set("user:" + userId, userFromDB);
        }

        return userFromDB;
    }
}
```

Redis Storage (After Running the Code)

sh

CopyEdit

```
127.0.0.1:6379> KEYS *
"user:1"

127.0.0.1:6379> GET user:1
"{\"id\":1,\"name\":\"Alice\",\"role\":\"ADMIN\"}"
```

🎯 Key Takeaways

✔ MySQL stores data as tables, while Redis stores data as key-value pairs.
✔ Spring Boot fetches MySQL data as Java objects and serializes them to store in Redis.
✔ Redis only stores binary or text-based data, so JSON serialization is required.
✔ When fetching from Redis, JSON is deserialized back into a Java object.
✔ This caching reduces database load and improves performance! 🚀

Would you like me to show **Redis caching for JWT tokens** as well? 😊

#Spring Boot **@Cacheable** relies on **RedisCacheManager**, which defaults to **JDK serialization** unless explicitly configured to use JSON serialization.

Even if RedisTemplate is configured with GenericJackson2JsonRedisSerializer, Spring's **caching mechanism does not automatically use it** unless you configure RedisCacheManager properly.

What Changes Does This Make?

- Uses **Jackson2JsonRedisSerializer** instead of `GenericJackson2JsonRedisSerializer`.
- Stores data in **readable JSON format**.
- Removes the unnecessary `@class` metadata.

Before (Unreadable JSON in Redis)

sh

CopyEdit

```
127.0.0.1:6379> GET users::1

"{\"@class\":\"com.example.cruddemo.entity.User\",\"id\":1,\"nam
e\":\"John Doe\",\"email\":\"john@example.com\"}"
```

After (Readable JSON in Redis)

sh

CopyEdit

```
127.0.0.1:6379> GET users::1

"{\"id\":1,\"name\":\"John
Doe\",\"email\":\"john@example.com\"}"
```

```
public class RedisConfig {

    @Bean
    public RedisTemplate<String, Object> redisTemplate(RedisConnectionFactory connectionFactory) {
        RedisTemplate<String, Object> template = new RedisTemplate<>();
        template.setConnectionFactory(connectionFactory);

        // Configure Jackson ObjectMapper for JSON serialization
        ObjectMapper objectMapper = new ObjectMapper();
        objectMapper.setVisibility(PropertyAccessor.ALL, JsonAutoDetect.Visibility.ANY);
        objectMapper.enable(SerializationFeature.INDENT_OUTPUT);
        objectMapper.disable(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES);

        Jackson2JsonRedisSerializer<Object> serializer = new Jackson2JsonRedisSerializer<>(Object.class);
        serializer.setObjectMapper(objectMapper);

        // Use JSON serialization for all keys and values
        template.setKeySerializer(new StringRedisSerializer());
        template.setValueSerializer(serializer);
        template.setHashKeySerializer(new StringRedisSerializer());
        template.setHashValueSerializer(serializer);

        template.afterPropertiesSet();
        return template;
    }
```

```java
@Bean
    public RedisCacheManager cacheManager(RedisConnectionFactory redisConnectionFactory) {
        ObjectMapper objectMapper = new ObjectMapper();
        objectMapper.setVisibility(PropertyAccessor.ALL, JsonAutoDetect.Visibility.ANY);
        objectMapper.enable(SerializationFeature.INDENT_OUTPUT);
        objectMapper.disable(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES);

        Jackson2JsonRedisSerializer<Object> serializer = new Jackson2JsonRedisSerializer<>(Object.class);
        serializer.setObjectMapper(objectMapper);

        RedisCacheConfiguration cacheConfig = RedisCacheConfiguration.defaultCacheConfig()
                .entryTtl(Duration.ofMinutes(30))  // Set TTL for cache
                .disableCachingNullValues()
                .serializeValuesWith(RedisSerializationContext.SerializationPair.fromSerializer(serializer));

        return RedisCacheManager.builder(redisConnectionFactory)
                .cacheDefaults(cacheConfig)
                .build();
    }
}
```

Redis Crud->

```
127.0.0.1:6379> KEYS *
1) "user::4"
2) "user::1"
3) "user::3"
4) "user::2"
5) "users::all"
127.0.0.1:6379> MGET user::1 user::2 user::3 user::4
1) "{\"@class\":\"com.example.cruddemo.entity.User\",\"id\":1,\"name\":\"John Doe\",\"email\":\"john@example.com\"}"
2) "{\"@class\":\"com.example.cruddemo.entity.User\",\"id\":2,\"name\":\"Anushka\",\"email\":\"anushka@example.com\"}"
3) "{\"@class\":\"com.example.cruddemo.entity.User\",\"id\":3,\"name\":\"Aarav\",\"email\":\"aarav@example.com\"}"
4) "{\"@class\":\"com.example.cruddemo.entity.User\",\"id\":4,\"name\":\"Aa\",\"email\":\"aa@example.com\"}"
```

Spring AOP-Aspect Oriented Programming

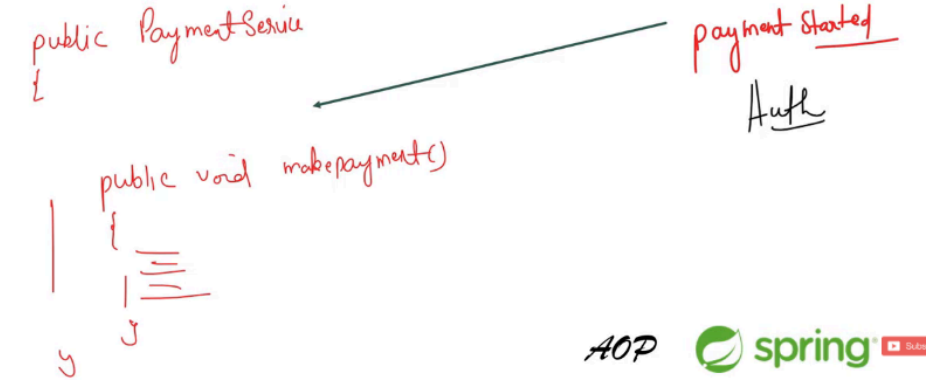
Beech ma koi naya service insert krna(these external services are called cross cutting concern) ha main service ma without modifying the actual code.

WHAT IS AOP ?

- AOP is a programming paradigm that aims to increase modularity by allowing the separation of cross-cutting concerns.
- Process of applying services or external services as transaction management or logging to our application classes without modifying the code.
- This services also called cross cutting concern.
- *Aspect-Oriented Programming* (AOP) complements Object-Oriented Programming (OOP) by providing another way of thinking about program structure.

Aspects->code jissa huma beech ma chalana ha .Apis which contain cross cutting concerns.

EXAMPLE



Payment krna ssa pehla-> authentication or payment started type kuch features lagaane ha

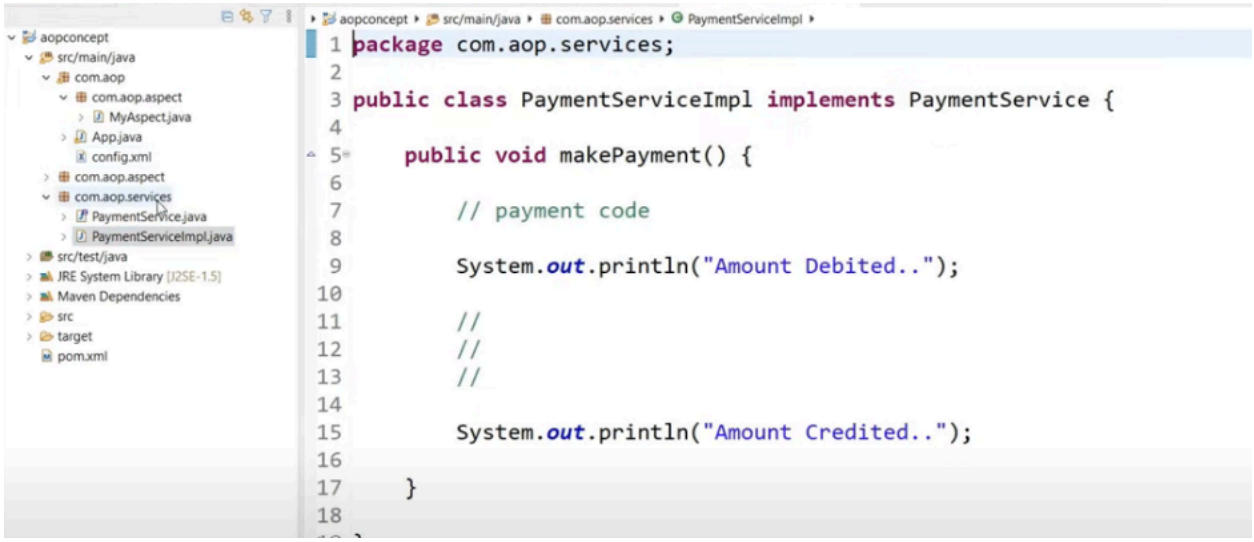
makePayement is joint point.JP are point of method execution.

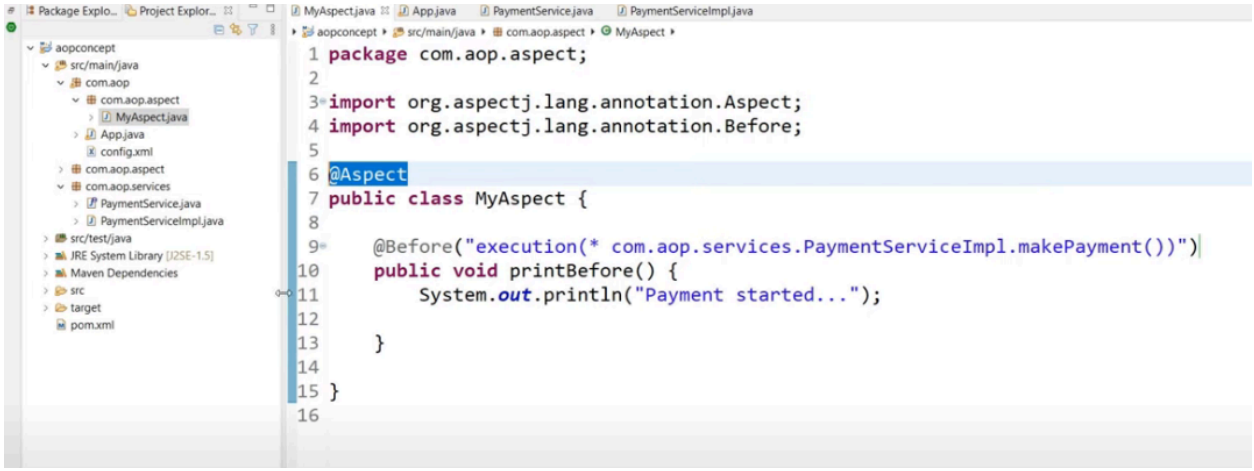
Advice->actions taken by Aspects at a particular joint point

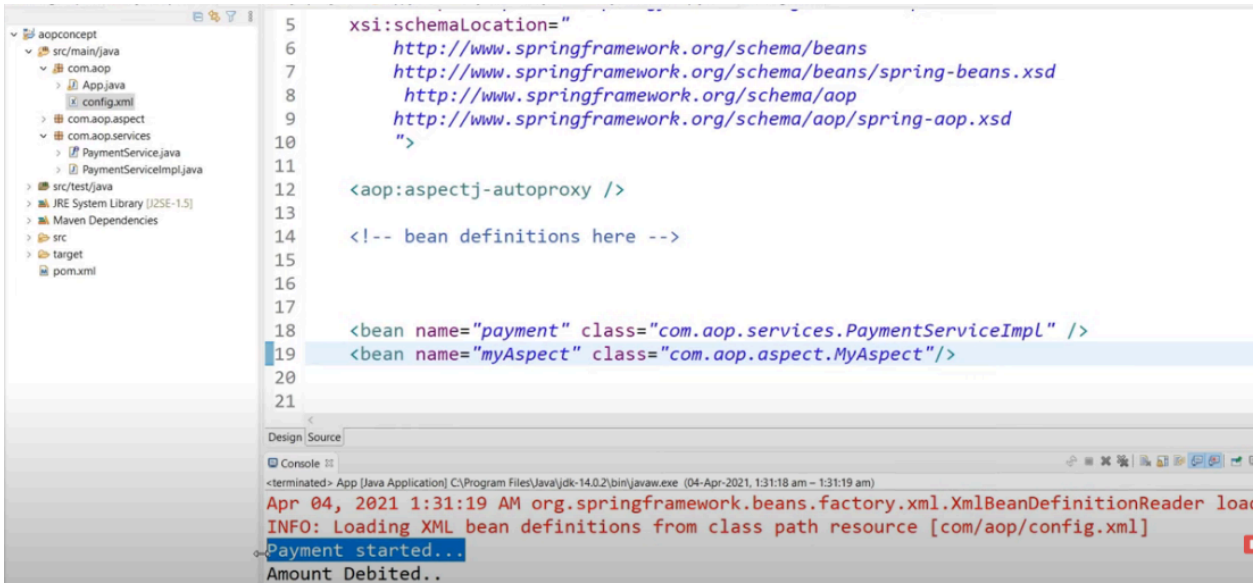
- **Aspect :**An aspect is a modularization of a concern that cuts across multiple classes.
- **Join point** :Join point is any point in your program such as method execution, exception handling, field access etc. Spring supports only method execution join point.
- **Advice** :Advice represent an action taken by an aspect at particular join point.
- **Pointcut** : It is an expression of AOP that marches join point.

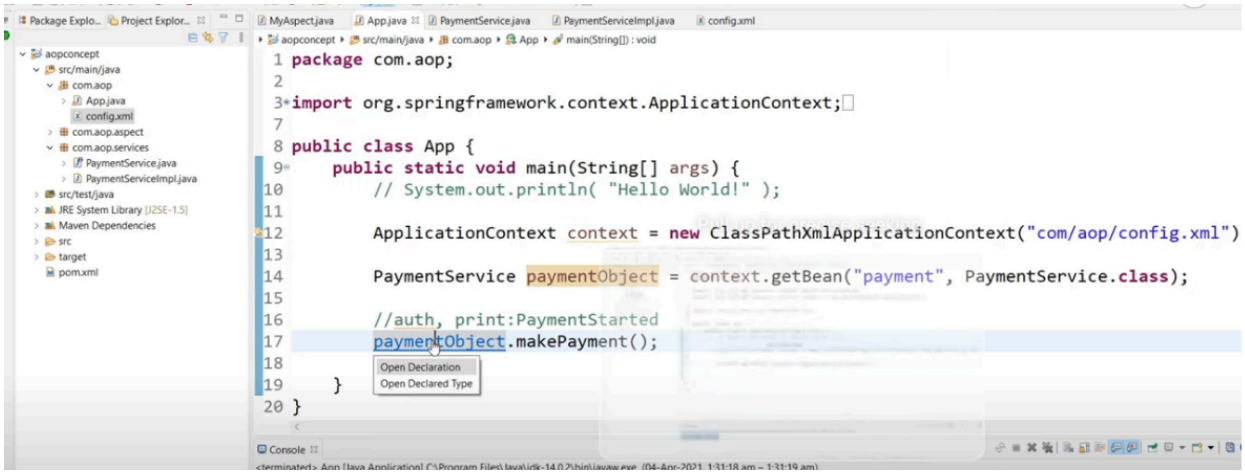
Advice are of many types->

1) After -method execution ka baad chalegi
2) Before-method execution ka pehla chalegi
3) After returning->successful
4) After throwing-> exception throw.




Package Explo...
Project Explor...
aopconcept
src/main/java
com.aop
com.aop.aspect
MyAspect.java
App.java
config.xml
com.aop.aspect
com.aop.services
PaymentService.java
PaymentServiceImpl.java
src/test/java
JRE System Library [J2SE-1.5]
Maven Dependencies
src
target
pom.xml
MyAspect.java
App.java
PaymentService.java
PaymentServiceImpl.java
aopconcept ▸ src/main/java ▸ com.aop.aspect ▸ MyAspect ▸
1 package com.aop.aspect;
2
3 import org.aspectj.lang.annotation.Aspect;
4 import org.aspectj.lang.annotation.Before;
5
6 @Aspect
7 public class MyAspect {
8
9 @Before("execution(* com.aop.services.PaymentServiceImpl.makePayment())")
10 public void printBefore() {
11 System.out.println("Payment started...");
12
13 }
14
15 }
16


aopconcept
src/main/java
com.aop
App.java
config.xml
com.aop.aspect
com.aop.services
PaymentService.java
PaymentServiceImpl.java
src/test/java
JRE System Library [J2SE-1.5]
Maven Dependencies
src
target
pom.xml
5 xsi:schemaLocation="
6 http://www.springframework.org/schema/beans
7 http://www.springframework.org/schema/beans/spring-beans.xsd
8 http://www.springframework.org/schema/aop
9 http://www.springframework.org/schema/aop/spring-aop.xsd
10 ">
11
12 <aop:aspectj-autoproxy />
13
14 <!-- bean definitions here -->
15
16
17
18 <bean name="payment" class="com.aop.services.PaymentServiceImpl" />
19 <bean name="myAspect" class="com.aop.aspect.MyAspect"/>
20
21
Design
Source
Console
<terminated> App [Java Application] C:\Program Files\Java\jdk-14.0.2\bin\javaw.exe (04-Apr-2021, 1:31:18 am – 1:31:19 am)
Apr 04, 2021 1:31:19 AM org.springframework.beans.factory.xml.XmlBeanDefinitionReader loa
INFO: Loading XML bean definitions from class path resource [com/aop/config.xml]
Payment started...
Amount Debited..



```
package com.aop;

import org.springframework.context.ApplicationContext;

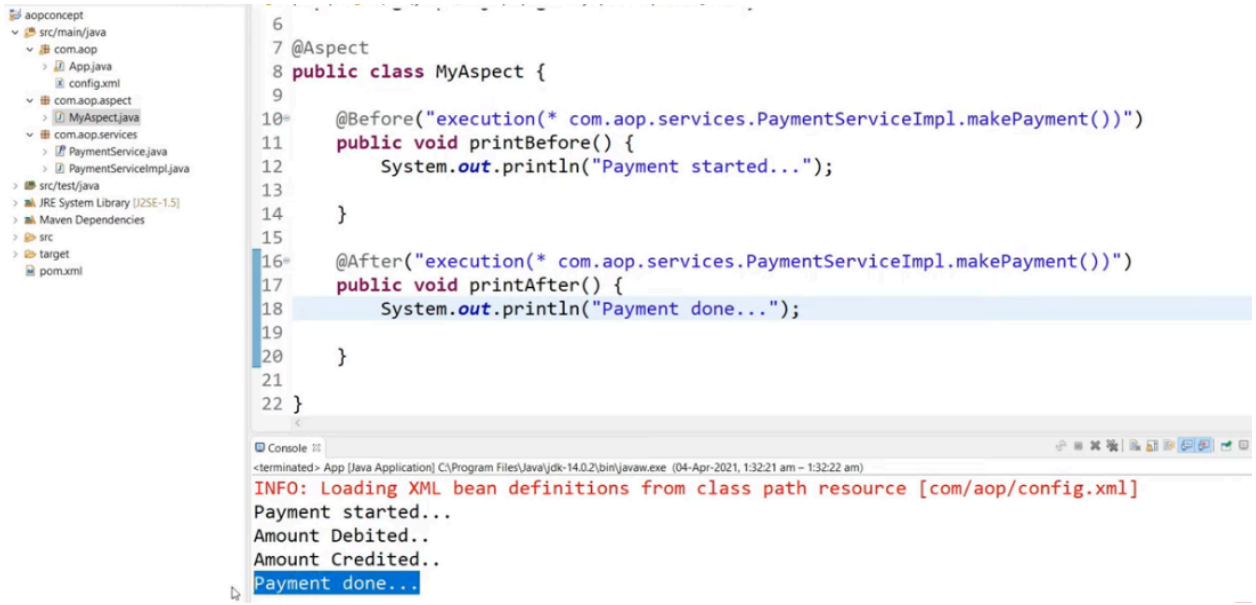
public class App {
    public static void main(String[] args) {
        // System.out.println( "Hello World!" );

        ApplicationContext context = new ClassPathXmlApplicationContext("com/aop/config.xml")

        PaymentService paymentObject = context.getBean("payment", PaymentService.class);

        //auth, print:PaymentStarted
        paymentObject.makePayment();

    }
}
```



```
@Aspect
public class MyAspect {

    @Before("execution(* com.aop.services.PaymentServiceImpl.makePayment())")
    public void printBefore() {
        System.out.println("Payment started...");

    }

    @After("execution(* com.aop.services.PaymentServiceImpl.makePayment())")
    public void printAfter() {
        System.out.println("Payment done...");

    }

}
```

```
INFO: Loading XML bean definitions from class path resource [com/aop/config.xml]
Payment started...
Amount Debited..
Amount Credited..
Payment done...
```



```
package com.aop.services;

public interface PaymentService {

    public void makePayment();

}
```

```
es.PaymentServiceImpl.makePayment(..))")
```

use .. if there is any argument in the funct to match the pattern.

Filters And Interceptors

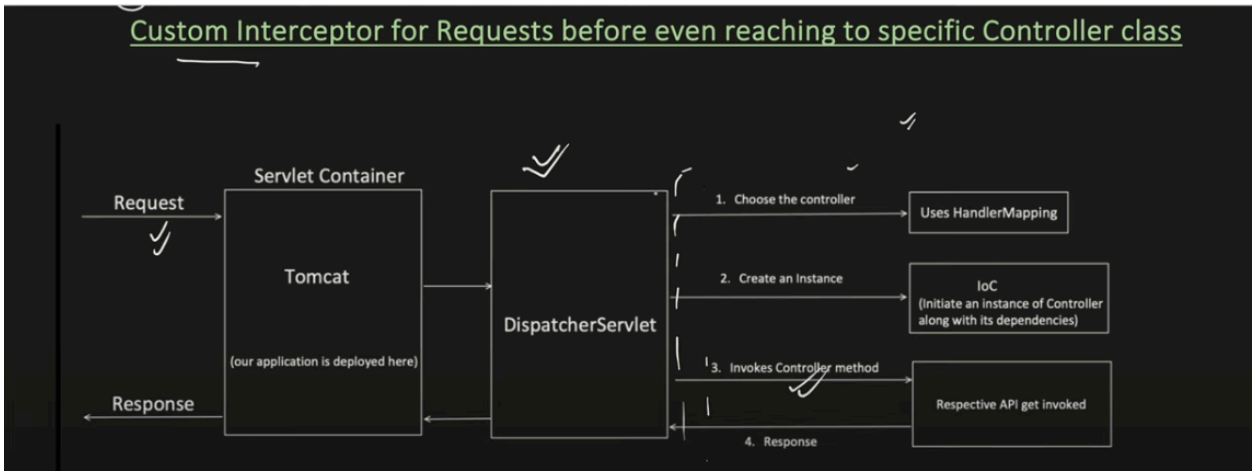
These are methods which are in the implementation of HandleInterceptor

-prehandle

- controller method will invoke

-post handle(if exception it will not run)

-aftercompletion(it is like finally whether exception or success it will run)





```java
@RestController
@RequestMapping(value = "/api/")
public class UserController {

    @Autowired
    User user;

    @GetMapping(path = "/getUser")
    public String getUser() {
        user.getUser();
        return "success";
    }
}
```

```java
@Component
public class MyCustomInterceptor implements HandlerInterceptor {

    @Override
    public boolean preHandle(HttpServletRequest request, HttpServletRes
        System.out.println("inside pre handle Method");
        return true;
    }

    @Override
    public void postHandle(HttpServletRequest request, HttpServletRespo
        System.out.println("inside post handle method");

    }

    @Override
    public void afterCompletion(HttpServletRequest request, HttpServlet
        System.out.println("inside after completion method");
    }
```




```java
@Configuration
public class AppConfig implements WebMvcConfigurer {

    @Autowired
    MyCustomInterceptor myCustomInterceptor;

    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(myCustomInterceptor)
                .addPathPatterns("/api/*") // Apply to these URL patterns
                .excludePathPatterns("/api/updateUser", "/api/deleteUser"); // Exclude
    }
}
```




http://localhost:8080/api/getUser

```
2024-08-31T23:01:49.217+05:30  INFO 4417 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServl
inside pre handle Method
hitting db to get the userdata
inside post handle method
inside after completion method
```

Step1: Creation of custom annotation

We can create Custom Annotation using keyword "@interface" java Annotation.

```
public @interface MyCustomAnnotation {
}
```

```
public class User {

    @MyCustomAnnotation
    public void updateUser(){
        //some business logic
    }
}
```

@Target :
this meta annotation, tells where we can apply the particular annotation on method or class or constructor etc.

```
@Target(ElementType.METHOD)
public @interface MyCustomAnnotation {
}
```

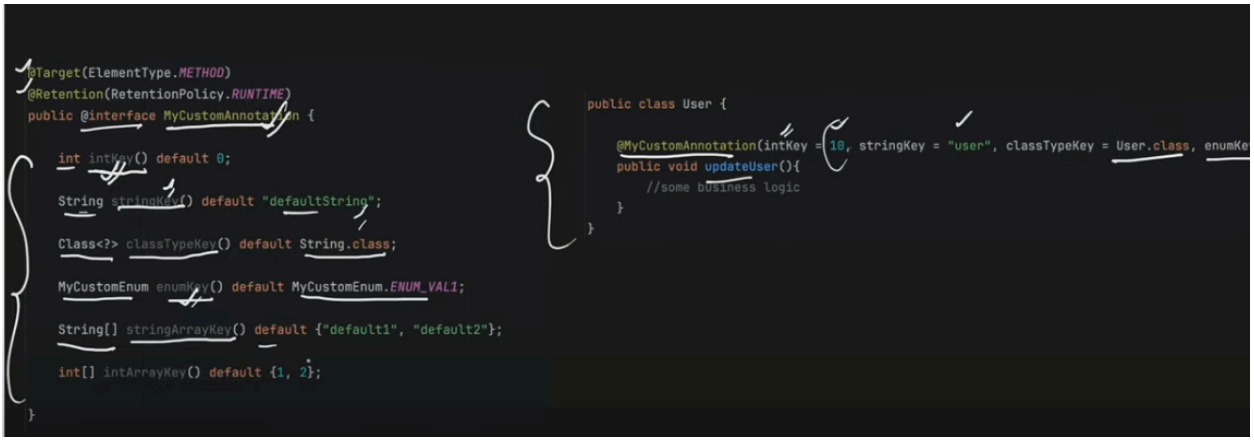
```
@Target({ElementType.CONSTRUCTOR, ElementType.METHOD, ElementType.PARAMETER, ElementType.FIELD})
public @interface MyCustomAnnotation {
}
```

@Retention:
this meta annotation tell, how the particular annotation will be stored in java.

RetentionPolicy.*SOURCE* :
Annotation will be discarded by compiler itself and its not even recorded in .class file.

RetentionPolicy.*CLASS* :
Annotation will be recorded in .class file but ignored by JVM during run time.

RetentionPolicy.*RUNTIME* :
Annotation will be recorded in .class file and also available during run time.



```java
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface MyCustomAnnotation {

    int intKey() default 0;

    String stringKey() default "defaultString";

    Class<?> classTypeKey() default String.class;

    MyCustomEnum enumKey() default MyCustomEnum.ENUM_VAL1;

    String[] stringArrayKey() default {"default1", "default2"};

    int[] intArrayKey() default {1, 2};

}
```

```java
public class User {

    @MyCustomAnnotation(intKey = 10, stringKey = "user", classTypeKey = User.class, enumKe
    public void updateUser(){
        //some business logic
    }
}
```

How to create CustomAnnotation with methods (more like a fields):

- No parameter, no body
- Return type is restricted to:
  - Primitive type (int, boolean, double etc.)
  - String
  - Enum
  - Class<?>
  - Annotations
  - Array of above types

```java
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface MyCustomAnnotation {

    String key() default "defaultKeyName";
}
```

```java
public class User {

    @MyCustomAnnotation(key = "userKey")
    public void updateUser(){
        //some business logic
    }
}
```

Step2: Creation of Custom Interceptor

```java
@Target({ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)
public @interface MyCustomAnnotation {
    String name() default "";
}
```

```java
@RestController
@RequestMapping(value = "/api/")
public class UserController {

    @Autowired
    User user;

    @GetMapping(path = "/getUser")
    public String getUser(){
        user.getUser();
        return "success";
    }
}
```

```java
@Component
public class User {

    @MyCustomAnnotation(name = "user")
    public void getUser() {
        System.out.println("get the user details");
    }
}
```

```java
@Component
@Aspect
public class MyCustomInterceptor {

    @Around("@annotation(com.conceptandcoding.learningspringboot.CustomInterceptor.MyCustomAnnotation)")
    public void invoke(ProceedingJoinPoint joinPoint) throws Throwable{ //advice

        System.out.println("do something before actual method");

        Method method = ((MethodSignature)joinPoint.getSignature()).getMethod();
        if(method.isAnnotationPresent(MyCustomAnnotation.class)){
            MyCustomAnnotation annotation = method.getAnnotation(MyCustomAnnotation.class);
            System.out.println("name from annotation: " + annotation.name());
        }

        joinPoint.proceed();
        System.out.println("do something after actual method");
    }
```

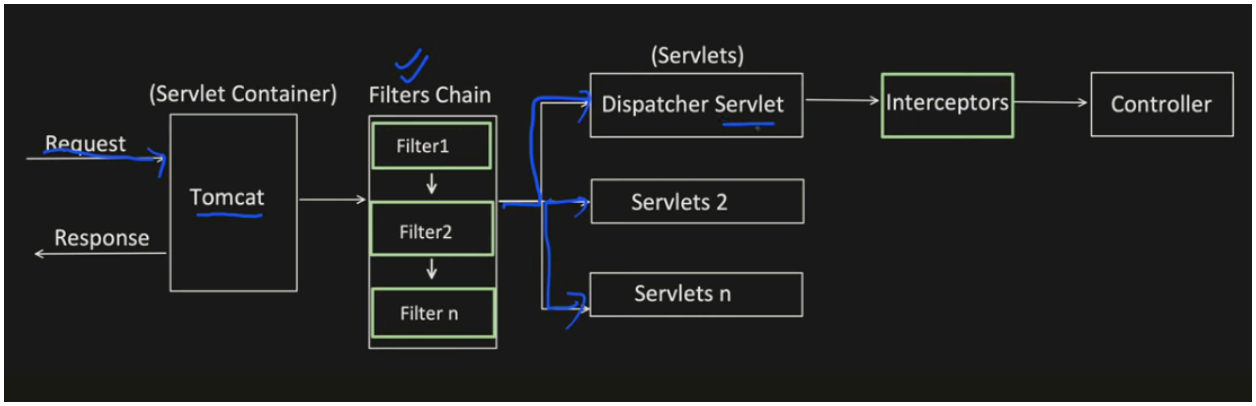
Output:

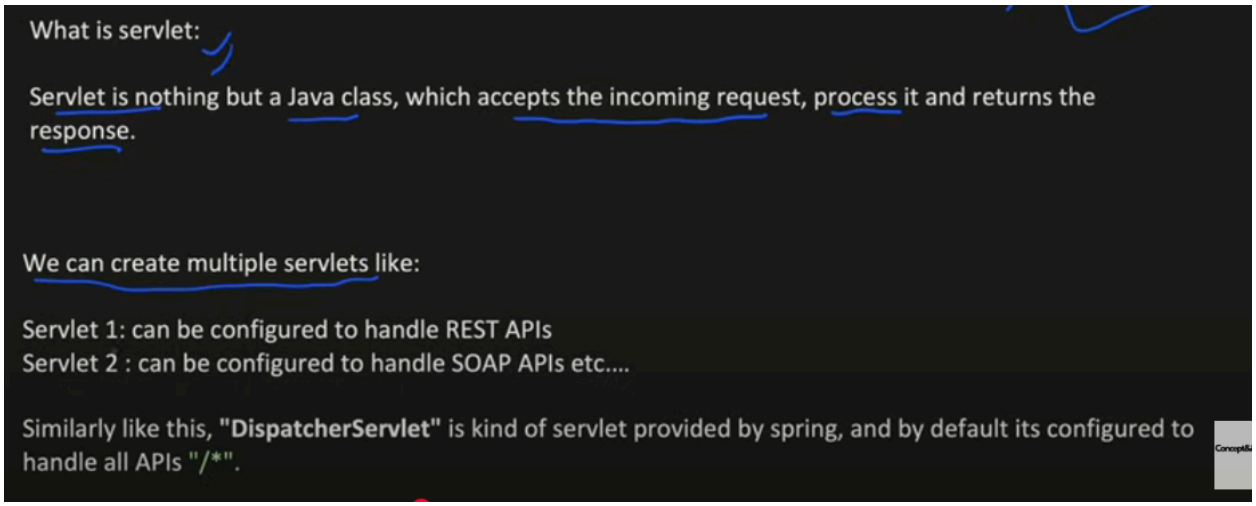
```
2024-08-31T17:51:51.344+05:30  INFO 3464 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet
do something before actual method
name from annotation: user
get the user details
do something after actual method
```

Filters and Interceptors:

Filter:
It intercept the HTTP Request and Response, before they reach to the servlet.

Interceptor:
Its specific to Spring framework, and intercept HTTP Request and Response, before they reach to the Controller. and often Servlet

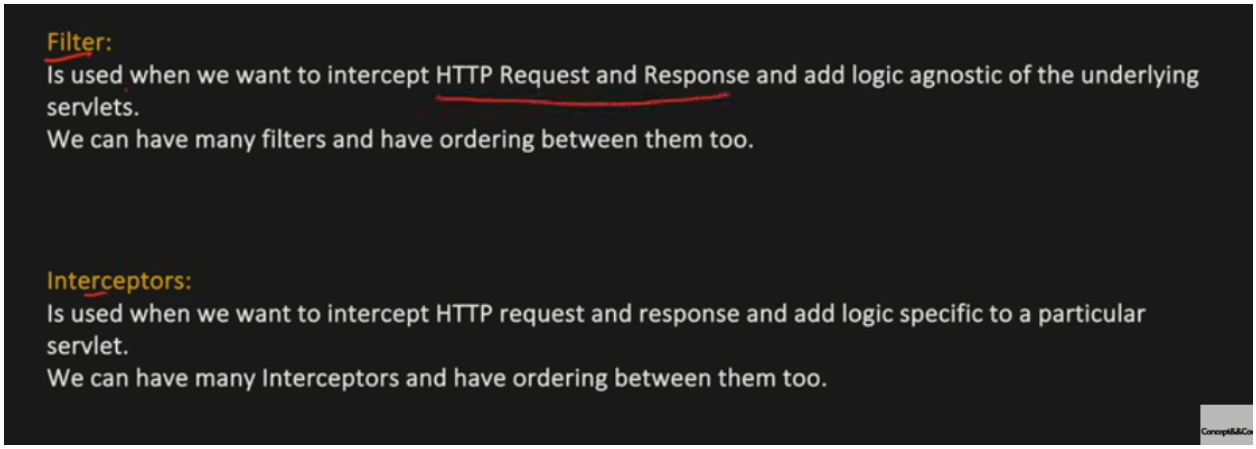




DispatcherServlet->very much related to SPring Boot,so interceptor is concept of spring boot only.

Filter when to use?->When I need to put the logic which is common to all the servlets like spring security(because it is applied on all the http requests

Interceptor-> when anything particular to the single servlet like restApi servlet



In previous video, we already saw, how to add 1 interceptor.
How "preHandle" , "postHandle" and "afterCompletion" comes into the picture.

Now here, will show how to add more than 1.

Also, if "preHandle" returns false, next filter and controller will not get invoked itself.

```java
public class AppConfig implements WebMvcConfigurer {

    @Autowired
    MyCustomInterceptor1 myCustomInterceptor1;

    @Autowired
    MyCustomInterceptor2 myCustomInterceptor2;


    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        //below order is maintained while calling interceptors
        registry.addInterceptor(myCustomInterceptor1)
                .addPathPatterns("/api/*") // Apply to these URL patterns
                .excludePathPatterns("/api/updateUser", "/api/deleteUser"); // Exclude for these URL patterns

        registry.addInterceptor(myCustomInterceptor2)
                .addPathPatterns("/api/*") // Apply to these URL patterns
                .excludePathPatterns("/api/updateUser");
    }
}
```

Output:

```
2024-09-02T17:01:11.719+05:30  INFO 7162 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet
2024-09-02T17:01:11.720+05:30  INFO 7162 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet
inside pre handle Method - MyCustomInterceptor1
inside pre handle Method - MyCustomInterceptor2
hitting db to get the userdata
inside post handle method- MyCustomInterceptor2
inside post handle method- MyCustomInterceptor1
inside after completion method - MyCustomInterceptor2
inside after completion method- MyCustomInterceptor1
```

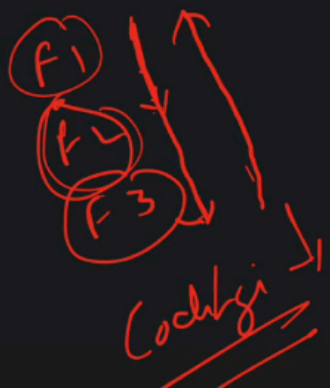
```java
import jakarta.servlet.*;
import java.io.IOException;

public class MyFilter1 implements Filter {
    @Override
    public void init(FilterConfig filterConfig) throws ServletException {
        Filter.super.init(filterConfig);
    }

    @Override
    public void doFilter(ServletRequest servletRequest, ServletResponse servletResponse, FilterChain filterChain)
            throws IOException, ServletException {

        System.out.println("MyFilter1 inside");
        filterChain.doFilter(servletRequest, servletResponse);
        System.out.println("MyFilter1 completed");
    }

    @Override
    public void destroy() {
        Filter.super.destroy();
    }
}
```




```java
import jakarta.servlet.*;
import java.io.IOException;

public class MyFilter2 implements Filter {
    @Override
    public void init(FilterConfig filterConfig) throws ServletException {
        Filter.super.init(filterConfig);
    }

    @Override
    public void doFilter(ServletRequest servletRequest, ServletResponse servl
            throws IOException, ServletException {

        System.out.println("MyFilter2 inside");
        filterChain.doFilter(servletRequest, servletResponse);
        System.out.println("MyFilter2 completed");
    }

    @Override
    public void destroy() {
        Filter.super.destroy();
    }
}
```

```java
@Configuration
public class AppConfig {

    @Bean
    public FilterRegistrationBean<MyFilter1> myFirstFilter() {
        FilterRegistrationBean<MyFilter1> filterRegistrationBean = new FilterRegistrationBean<>();
        filterRegistrationBean.setFilter(new MyFilter1());
        filterRegistrationBean.addUrlPatterns("/*");
        filterRegistrationBean.setOrder(2);
        return filterRegistrationBean;
    }

    @Bean
    public FilterRegistrationBean<MyFilter2> mySecondFilter() {
        FilterRegistrationBean<MyFilter2> filterRegistrationBean = new FilterRegistrationBean<>();
        filterRegistrationBean.setFilter(new MyFilter2());
        filterRegistrationBean.addUrlPatterns("/*");
        filterRegistrationBean.setOrder(1);
        return filterRegistrationBean;
    }
}
```

Output:

```
2024-09-02T17:30:39.279+05:30  INFO 7350 --- [nio-8080-exec-1]
2024-09-02T17:30:39.280+05:30  INFO 7350 --- [nio-8080-exec-1]
MyFilter2 inside
MyFilter1 inside
hitting db to get the userdata
MyFilter1 completed
MyFilter2 completed
```

If both Interceptor and Filter used together, Output will look like this.

```
2024-09-02T17:32:25.643+05:30  INFO 7373 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet
2024-09-02T17:32:25.644+05:30  INFO 7373 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet
MyFilter1 inside
MyFilter2 inside
inside pre handle Method - MyCustomInterceptor1
inside pre handle Method - MyCustomInterceptor2
hitting db to get the userdata
inside post handle method- MyCustomInterceptor2
inside post handle method- MyCustomInterceptor1
inside after completion method - MyCustomInterceptor2
inside after completion method- MyCustomInterceptor1
MyFilter2 completed
MyFilter1 completed
```